TOPEET 迅为

www.topeetboard.com

「新一代 AIOT 设备平台的优秀典范」

超长供货周期　　7 X 24 小时稳定运行

软硬件全开源　　丰富接口和高扩展性

iTOP-RK3568

迅为电子团队编著

官方微信公众号　　迅为商城

010-8527-0708

www.topeetboard.com

Beijing TOPEET Electronics Co.Ltd

更新记录

更新版本	修改内容
V1.0	初版

目录

更新记录 2
目录 3
注意事项与售后维修 5
技术支持与开发定制 7
资料获取与后续更新 9
第 1 章 i2c 使用 10
1.1 i2c 通讯 10
1.2 定义和注册 I2C 设备 11
1.3 定义和注册 I2C 驱动 11
1.3.1 定义 I2C 驱动 12
1.3.2 注册 I2C 驱动 13
1.3.3 通过 I2C 收发数据 13
1.4 i2C Tools 14
1.5 常见问题 14
第 2 章 SPI 使用 16
2.1 RK3568 功能特点 16
2.2 SPI 代码路径 17
2.3 RK3568 做 SPI 从设备 17
2.4 RK3568 做 SPI 主设备 19
2.5 SPI 驱动 21
2.6 SPI 回环测试 23
2.6.1 配置驱动 23
2.6.2 修改设备树 24
2.6.3 回环测试 24
2.7 常见问题 25
第 3 章 ADC 使用 27
3.1 设备树配置 27
3.2 在驱动文件中匹配 DTS 节点 28
3.3 驱动说明 31
3.4 调试方法 33
第 4 章 LED 使用 34
4.1 设备树编写 34
4.2 LED 子系统 35
4.2.1 用户态 35
4.2.2 内核驱动 35
4.3 LED 驱动 37
4.4 使用方法 43
第 5 章 UART 使用 45
5.1 设备树配置 45
5.2 测试 45
5.3 调试串口配置为普通串口 45

5.4 串口回环测试 46
第 6 章 IR 使用 48
6.1 修改内核空间 48
6.2 用户空间测试 50
第 7 章 Ethernet 使用 51
7.1 设备树配置 51
7.2 修改网口灯状态 52
第 8 章 Camera 使用 55
8.1 MIPI_CSI 用法 55
8.2 Full mode 配置 55
8.2.1 设备树配置 55
8.3 Split Mode 配置 59
8.3.1 设备树配置 59
8.4 软件相关目录 59
第 9 章 PWM 使用 61
9.1 配置设备树 61
9.2 配置内核驱动 61
9.3 PWM 调试 62
9.4 PWM 使用 62
9.5 常见问题 64
第 10 章 RTC 使用 65
10.1 设备树驱动配置 65
10.2 接口使用 66
第 11 章 CAN 使用 67
11.1 CAN 驱动配置 67
11.2 设备树节点配置 68
11.3 canutils 工具配置使用 69
11.4 CAN 应用测试程序 70

注意事项与售后维修

一 注意事项

◆ 请注意和遵循标注在产品上的所有警示和指引信息；

◆ 请勿带电插拔核心板及外围模块；

◆ 使用产品之前，请仔细阅读本手册，并妥善保管，以备将来参考；

◆ 请使用配套电源适配器，以保证电压、电流的稳定；

◆ 请在凉爽、干燥、清洁的地方使用本产品；

◆ 请勿在冷热交替环境中使用本产品，避免结露损坏元器件；

◆ 请勿将任何液体泼溅在本产品上，如果不慎被任何液体泼溅或浸润，请立刻断电并充分晾干；

◆ 请勿使用有机溶剂或腐蚀性液体清洗本产品；

◆ 请勿在多尘、脏乱的环境中使用本产品，如果长期不使用，请包装好本产品；

◆ 请勿在震动过大的环境中使用，任何跌落、敲打或剧烈晃动都可能损坏线路及元器件；

◆ 请勿在通电情况下，插拔核心板及外围模块(特别是串口模块)；

◆ 请勿自行维修、拆解本产品，如产品出现故障应及时联系本公司进行维修；

◆ 请勿自行修改或使用未经授权的配件，由此造成的损坏将不予保修；

二 售后维修

凡是通过迅为直接购买或经迅为授权的正规代理商处购买的迅为产品，均可享受以下权益：

1、开发板本身 1 年免费保修服务（配件除外）；

2、保修期满后出现产品异常，迅为提供有偿维修服务，可与迅为取得联系，收费视具体情况而定。如遇损坏程度严重等其他不可控因素导致无法维修的，公司不再提供维修服务；

3、如您购买的产品需要维修或检测，请提前备份机器内的相关数据。迅为不对因数据丢失所造成的损失负责。

注：以下情况不属于免费维修范围，可提供有偿维修：

1、超出保修期的产品；

2、非保元件：CPU、内存芯片、Flash；

3、由于使用不当，出现诸如 PCB 烧毁、破裂等物理损伤的产品；

4、由于人为疏忽或错误使用、未按说明书规定使用而造成的产品损坏等；

5、拆装或更换组件、器件而造成无法复原的开发板；

6、在将故障件返回迅为技术服务部的过程中由于包装或运输操作不当造成损坏的产品。

维修周期：收到返修产品后，我们将即日安排工程师进行检测，我们将在短时间内维修或更换并寄回。一般的故障维修周期为 5 个工作日（自我司收到物品之日起，不计运输过程时间），由于特殊故障导致无法短期内维修的产品，我们会与用户另行沟通并确认维修周期。

维修费用：在免费保修期内的产品，由于产品质量问题引起的故障，不收任何维修费用；不属于免费保修范围内的故障或损坏，在检测确认问题后，我们将与客户沟通并确认维修费用，我们仅收取元器件材料费，不收取维修服务费；超过保修期限的产品，根据实际损坏的程度来确定收取的元器件材料费和维修服务费。

运输费用：产品正常保修时，用户寄回的运费由用户承担，维修后寄回给用户的费用由我司承担。非正常保修产品来回运费均由用户承担。

联系方式：

地　址：北京市海淀区永翔北路 9 号中国航发大厦三层北京迅为电子有限公司

联系人：迅为开发板售后服务部 北京迅为电子有限公司

邮　编：100094

电　话：010-58957586

技术支持与开发定制

1、技术支持范围

(1) 了解产品的软、硬件资源提供情况咨询；
(2) 产品的软、硬件手册使用过程中遇到的问题；
(3) 下载和烧写更新系统过程中遇到的问题；
(4) 产品用户的资料丢失、更新后重新获取；
(5) 产品的故障判断及售后维修服务。

2、技术讨论范围

由于嵌入式系统知识范围广泛，涉猎种类繁多，我们无法保证对各种问题都能一一解答，以下内容无法供技术支持，只能提供建议。

(1) 源码如何理解和修改，电路板的自行设计制作或修改；
(2) 如何编译和移植操作系统；
(3) 用户在自行修改以及开发中遇到的软硬件问题。

3、技术支持方式

论坛：http://bbs.topeetboard.com/forum.php
电话：0312-6796610
邮箱：support@topeetboard.com

4、技术支持服务时间

上午 9:00--12:00,下午 13:30--17:30(周一至周六）
QQ 群主动技术支持：
上午 10:00--11:00,下午 15:00--16:00(周一至周六）

5、定制开发服务

本公司提供嵌入式操作系统底层驱动、硬件板卡的有偿定制开发服务，以缩短您的产品开

发周期。请将需求：

发送邮件到： support@topeetboard.com

联系电话: 0312-6796610

淘宝店铺 1：

https://arm-board.taobao.com/?spm=a1z10.1-c-s.0.0.7bf93dd3q2C808

淘宝店铺 2：

https://shop459378556.taobao.com/shop/view_shop.htm?spm=a230r.1.14.48.25b84a81oPy9vl&user_number_id=2207480684984

资料获取与后续更新

一. 资料的获取

(1) 百度网盘下载

网盘的链接在购买开发板后可以在迅为电子技术支持 QQ 群下载。如果链接有更新，会在群里贴通告

二 后续更新

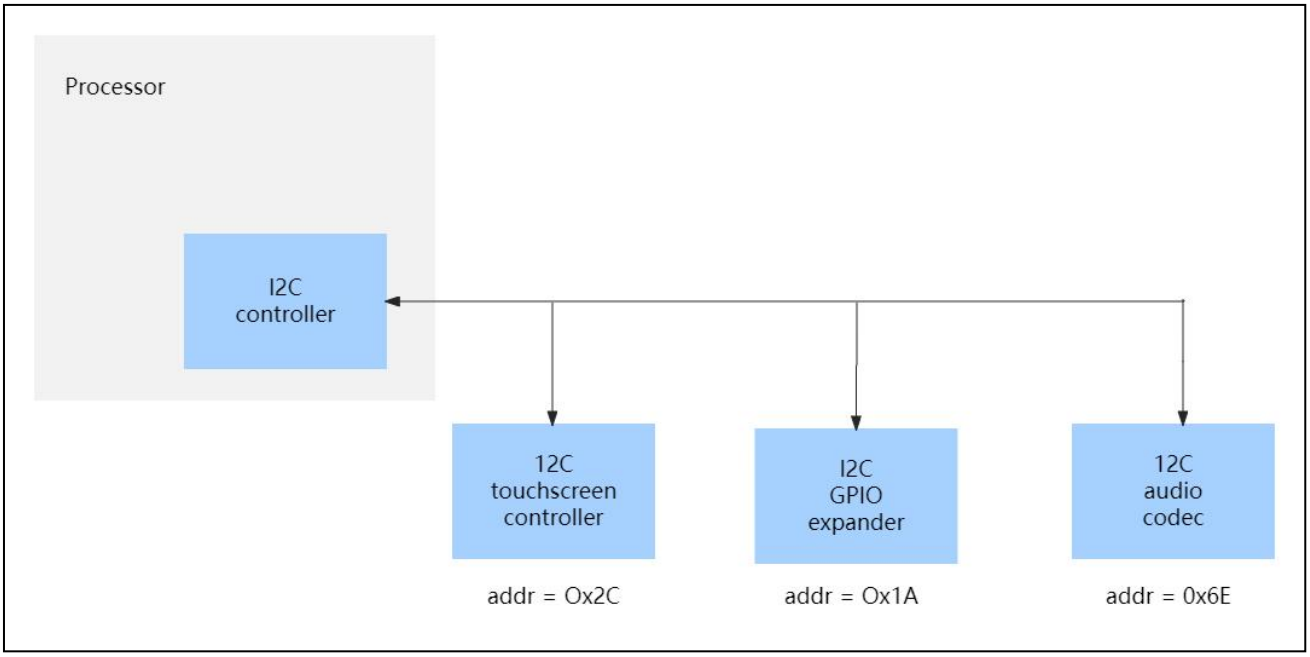
后续文档、视频等资料的更新，为了确保您的资料是最新状态，请密切关注我们的动态，我们将会通过微信公众号和 QQ 群推送。关注“迅为电子”微信公众号，不定期分享教程、资料和行业干货及产品一线资料。

迅为电子
让学习更容易，让开发更简单

第 1 章 i2c 使用

1.1 i2c 通讯

I2c(IIC)协议是在电子设备中常见的通讯协议，通过 I2c，我们可以对各种各样的 I2C 器件进行控制，一般受控的设备被称为从机，比如姿态传感器 MPU6050，温度图像传感器 MLX90640 等等，都是通过 I2C 协议和开发板进行通讯的。关于 i2C 协议的具体内容，在此不做详解，感兴趣的可以自行搜索学习。



iTOP-RK3568 开发板上有 6 个片上 i2c 控制器，各个 i2c 信息如下图所示：（可以查看 07 硬件手册）

功能	信号网络标号	I/O	默认功能	BALL
I2C0	I2C0_SCL_PMIC	O	I2C0 时钟	AF24
	I2C0_SDA_PMIC	I/O	I2C0 数据	AB21
I2C1	I2C1_SCL_TP	O	I2C1 时钟	AG24
	I2C1_SDA_TP	I/O	I2C1 数据	AB20
I2C2	I2C2_SCL_M1	O	I2C2 时钟	V5
	I2C2_SDA_M1	I/O	I2C2 数据	V6
I2C3	I2C3_SCL_M0	O	I2C3 时钟	E18
	I2C3_SDA_M0	I/O	I2C3 数据	D18
I2C4	I2C4_SDA_M1	I/O	I2C4 数据	D26
	I2C4_SCL_M1	O	I2C4 时钟	E25
I2C5	I2C5_SCL_M0	O	I2C5 时钟	AF1
	I2C5_SDA_M0	I/O	I2C5 数据	AE1

本文主要描述如何在 iTOP-RK3568 开发板上配置 I2C。配置 I2C 分为两大步骤。第一步是定义和注册 I2C 设备，第二步是定义和注册 I2C 驱动。

下面配置以触摸芯片 ft5x06 为例。

1.2 定义和注册 I2C 设备

在注册 I2C 设备时，需要结构体 i2c_client 来描述 I2C 设备。在标准 Linux 中，用户只需要提供相应的 I2C 设备信息，Linux 就会根据所提供的信息构造 i2c_client 结构体。

I2C 设备信息以节点的形式写到设备树文件中，如下图所示：

```
&i2c1 {
    status = "okay";
    ft5x061:ft5x06@38 {
            status = "okay";
            compatible = "edt,edt-ft5306";
            reg = <0x38>;
            touch-gpio = <&gpio3 RK_PA5 IRQ_TYPE_EDGE_RISING>;
            interrupt-parent = <&gpio3>;
            interrupts = <RK_PA5 IRQ_TYPE_LEVEL_LOW>;
            reset-gpio = <&gpio0 RK_PB6 GPIO_ACTIVE_LOW>;
            touchscreen-size-x = <800>;
            touchscreen-size-y = <1280>;
            touch_type = <1>;
    };
};
```

1.3 定义和注册 I2C 驱动

在上个小节中已经在设备树中定义和注册了 I2C 设备，接下来注册和定义 I2C 设备驱动。在 Linux 内核中，某些 I2C 从设备的驱动是默认自带的，有些是不带的。在内核源码目录下输入“grep -rn “XXX””” XXX 指设备树中对应的 compatible 值。如果可以查找到.c 文件说明此文件是该设备对应的驱动。如果查找不到，需要在内核合适的位置添加驱动文件，Makefile，Kconfig，将驱动文件编译进 Linux 内核中。

例如，我们找到 edt-ft5x06.c 文件，是 FT5x06 芯片的驱动文件，如下图所示：

```
topeet@ubuntu:~/Linux/rk356x_linux/kernel$ grep -rn "edt,edt-ft5306"
匹配到二进制文件 .tmp_vmlinux1
匹配到二进制文件 kernel.img
匹配到二进制文件 vmlinux.o
匹配到二进制文件 zboot.img
匹配到二进制文件 boot.img
匹配到二进制文件 vmlinux
匹配到二进制文件 resource.img
匹配到二进制文件 .tmp_vmlinux2
drivers/input/touchscreen/edt-ft5x06.c:1174:    { .compatible = "edt,edt-ft5306", .data = &edt_ft5x06_data },
匹配到二进制文件 drivers/input/touchscreen/edt-ft5x06.o
arch/arm/boot/dts/am437x-sk-evm.dts:601:                compatible = "edt,edt-ft5306", "edt,edt-ft5x06";
arch/arm64/boot/dts/rockchip/topeet_rk3568_lcds.dtsi:302:                              compatible = "edt,edt-ft5306";
arch/arm64/boot/dts/rockchip/topeet_rk3568_lcds.dtsi:320:                              compatible = "edt,edt-ft5306";
匹配到二进制文件 arch/arm64/boot/dts/rockchip/rk3568-evb1-ddr4-v10-linux.dtb
arch/arm64/boot/dts/rockchip/.rk3568-evb1-ddr4-v10-linux.dtb.dts.tmp:9174:    compatible = "edt,edt-ft5306";
arch/arm64/boot/dts/rockchip/.rk3568-evb1-ddr4-v10-linux.dtb.dts.tmp:9192:    compatible = "edt,edt-ft5306";
匹配到二进制文件 arch/arm64/boot/Image
Documentation/devicetree/bindings/input/touchscreen/edt-ft5x06.txt:19:          or:  "edt,edt-ft5306"
topeet@ubuntu:~/Linux/rk356x_linux/kernel$
```

接下来我们在内核配置 menuconfig 中选中 ft5x06 驱动，重新编译内核，烧写内核镜像之后便支持 FT5x06 芯片了。

下面我们分析下 edt-ft5x06.c 文件。

1.3.1 定义 I2C 驱动

在定义 i2C 驱动之前，用户首先要定义变量 of_device_id 和 i2c_device_id。

of_device_id 用于在驱动中调用设备树文件中定义的设备信息，定义如下图所示：

```c
static const struct of_device_id edt_ft5x06_of_match[] = {
    { .compatible = "edt,edt-ft5206", .data = &edt_ft5x06_data },
    { .compatible = "edt,edt-ft5306", .data = &edt_ft5x06_data },
    { .compatible = "edt,edt-ft5406", .data = &edt_ft5x06_data },
    { .compatible = "edt,edt-ft5506", .data = &edt_ft5506_data },
    /* Note focaltech vendor prefix for compatibility with ft6236.c */
    { .compatible = "focaltech,ft6236", .data = &edt_ft6236_data },
    { /* sentinel */ }
};
```

定义了变量 i2c_device_id，如下图所示：

```c
static const struct i2c_device_id edt_ft5x06_ts_id[] = {
    { .name = "edt-ft5x06", .driver_data = (long)&edt_ft5x06_data },
    { .name = "edt-ft5506", .driver_data = (long)&edt_ft5506_data },
    /* Note no edt- prefix for compatibility with the ft6236.c driver */
    { .name = "ft6236", .driver_data = (long)&edt_ft6236_data },
    { /* sentinel */ }
};
```

i2c_driver 如下图所示：

```
static struct i2c_driver edt_ft5x06_ts_driver = {
    .driver = {
        .name = "edt_ft5x06",
        .of_match_table = of_match_ptr(edt_ft5x06_of_match),
        .pm = &edt_ft5x06_ts_pm_ops,
    },
    .id_table = edt_ft5x06_ts_id,
    .probe    = edt_ft5x06_ts_probe,
    .remove   = edt_ft5x06_ts_remove,
};
```

注：变量 id_table 指示该驱动所支持的设备。

1.3.2 注册 I2C 驱动

我们经常用 int i2c_register_driver 函数和 i2c_add_driver 宏来添加 i2c 设备驱动，可以用下述的宏简写上述的俩个 API，参数是已经填充好的 struct i2c_driver 变量;

```
#define module_i2c_driver(__i2c_driver) \
        module_driver(__i2c_driver, i2c_add_driver, \
                      i2c_del_driver)
```

edt-ft5x06.c 文件中，如下所示：

```
module_i2c_driver(edt_ft5x06_ts_driver);
```

在注册 i2c 设备驱动的时候，会遍历 i2c 设备，如果该驱动支持所遍历到的设备，则会调用该驱动的 probe 函数。

1.3.3 通过 I2C 收发数据

注册好 i2c 驱动之后，即可进行 i2c 通讯。向从机读取发送信息，如下所示：

```
static int edt_ft5x06_ts_readwrite(struct i2c_client *client,
                                   u16 wr_len, u8 *wr_buf,
                                   u16 rd_len, u8 *rd_buf)
{
    struct i2c_msg wrmsg[2];
    int i = 0;
    int ret;
    if (wr_len) {
        wrmsg[i].addr  = client->addr;
        wrmsg[i].flags = 0;
```

```
            wrmsg[i].len = wr_len;
            wrmsg[i].buf = wr_buf;
            i++;
        }
        if (rd_len) {
            wrmsg[i].addr    = client->addr;
            wrmsg[i].flags = I2C_M_RD;
            wrmsg[i].len = rd_len;
            wrmsg[i].buf = rd_buf;
            i++;
        }
        ret = i2c_transfer(client->adapter, wrmsg, i);
        if (ret < 0)
            return ret;
        if (ret != i)
            return -EIO;
        return 0;
    }
```

1.4 i2C Tools

I2C tool 是一个开源工具，需要下载源码交叉编译，下载地址是：https://www.kernel.org/pub/software/utils/i2c-tools/ 编译之后会生成 i2cdetect，i2cdump,i2cset,i2cget 等工具，可以直接在命令行上进行调试。

- i2cdetect--用来列举 I2C bus 和上面所有的设备
- i2cdump--显示 i2c 设备所有寄存器的值
- i2cget--读取 i2c 设备某个寄存器的值
- i2cset--写入 i2c 设备某个寄存器的值

1.5 常见问题

（1）在 linux 内核中，已经实现了 i2c 用 GPIO 模拟，但是一般不推荐使用 GPIO，效率不高。

（2）调用 i2c_transfer 返回值是-6 说明是 NACK 错误，即从设备无相应应答，可能是以下几种情况：

- I2C 地址错误；
- I2C slave 设备处于不正常工作状态，比如没有上电，错误的上电时序以及设备异常等；
- I2C 时序不符合 slave 设备所要求也会产生 NACK 信号，比如 slave 设备需要的是 stop 信号,而不是 repeat start 信号的时候；
- I2C 总线受外部干扰导致的，用示波器测量可以看到是一个 ACK 波形。

（3）当出现 I2C 的 log："timeout, ipd: 0x00, state: 1"时，此时 I2C 控制器工作异常，无法产生中断状态，start 时序无法发出，有以下几种可能：

- I2C SCL 或者 SDA Pin 脚 iomux 错误；
- I2C 的上拉电压不对，如电压不够或者上拉电源没有等；
- I2C Pin 脚被外设拉住，电压不对；
- I2C 时钟未开，或者时钟源太小；
- I2C 同时配置了 CON_START 和 CON_STOP 位。

（4）当出现 I2C 的 log："timeout, ipd: 0x10, state: 1"时，此时 I2C 控制器工作正常，但是 cpu 无法响应 I2C 中断，此时可能 cpu0 被阻塞了（一般 I2C 中断都在 cpu0 上面，通过 cat /proc/interrups 可以查看），或者可能是 I2C 中断位被关闭了。

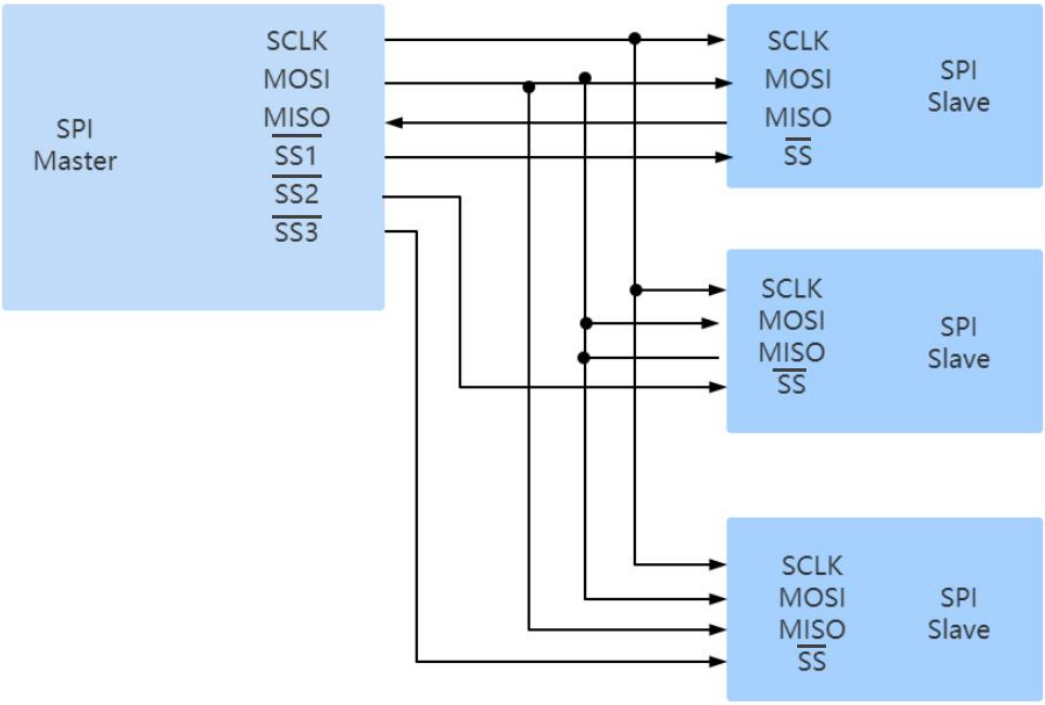
（5）当出现 I2C 的 log 类似："timeout, ipd: 0x80, state: 1"时，看到 ipd 为 0x80 打印，可以说明当前 SCL 被 slave 拉住，要判断被哪个 slave 拉住：

一是排除法，适用于外设不多的情况，而且复现概率高；

二是需要修改硬件，在 SCL 总线上串入电阻，通过电阻两端产生的压差来确定，电压更低的那端外设为拉低的 slave，电阻的选取以不影响 I2C 传输且可以看出压差为标准，一般上拉电阻的 1/20 以上都可以，如果是 host 拉低也可以看出。另外在此基础上通过示波器来抓取波形更加直观，比较不同 slave 和 host 的低电平大小，与最后出问题时的低电平大小比较，相等的就是拉低总线的“元凶”。常见的情况是 sda 被拉低，证明是谁拉低的，同样参考上面“SCL 被拉低" 的方法两种。

第 2 章 SPI 使用

在前面的章节，我们学习了 i2c 通讯协议使用相关的内容，本章节我们接着为大家介绍另一种通讯协议的使用——SPI 通讯协议。SPI 通讯协议也是电子器件通讯中常见的一种通讯协议。SPI 的通信协议的通信速率要更高，因为 SPI 通信采用的是全双工的通讯方式。比如一些小尺寸的 LCD 屏幕，AD 转换芯片，RFID 模块，SPI-FLASH 等等对通信速率有要求的器件上，一般会采用 SPI 接口。关于 SPI 协议的具体内容，在此不做详细解释，大家可以自行学习。SPI 设备之间使用全双工模式通信，是一个主机和一个或者多个从机的主从模式。主机负责初始化帧，这个数据传输帧可以用于读写俩种操作，片选线路可以从多个从机选择一个来响应主机的请求。



2.1 RK3568 功能特点

以下是 RK3568 SPI 驱动支持的一些特性。

- 默认采用摩托罗拉 SPI 协议
- 支持 8 位和 16 位
- 软件可编程时钟频率和传输速率高达 50MHz
- 支持 SPI 4 种传输模式配置

- 每个 SPI 控制器支持一个到 2 个片选
- 框架支持 slave 和 master 俩种模式

SPI 以主从方式工作，这种模式通常有一个主设备和一个或多个从设备，需要至少 4 根线，分别是：

CS	片选信号
SCLK	时钟信号
MOSI	主设备数据输出，从设备数据输入
MISO	主设备数据输入，从设备数据输出

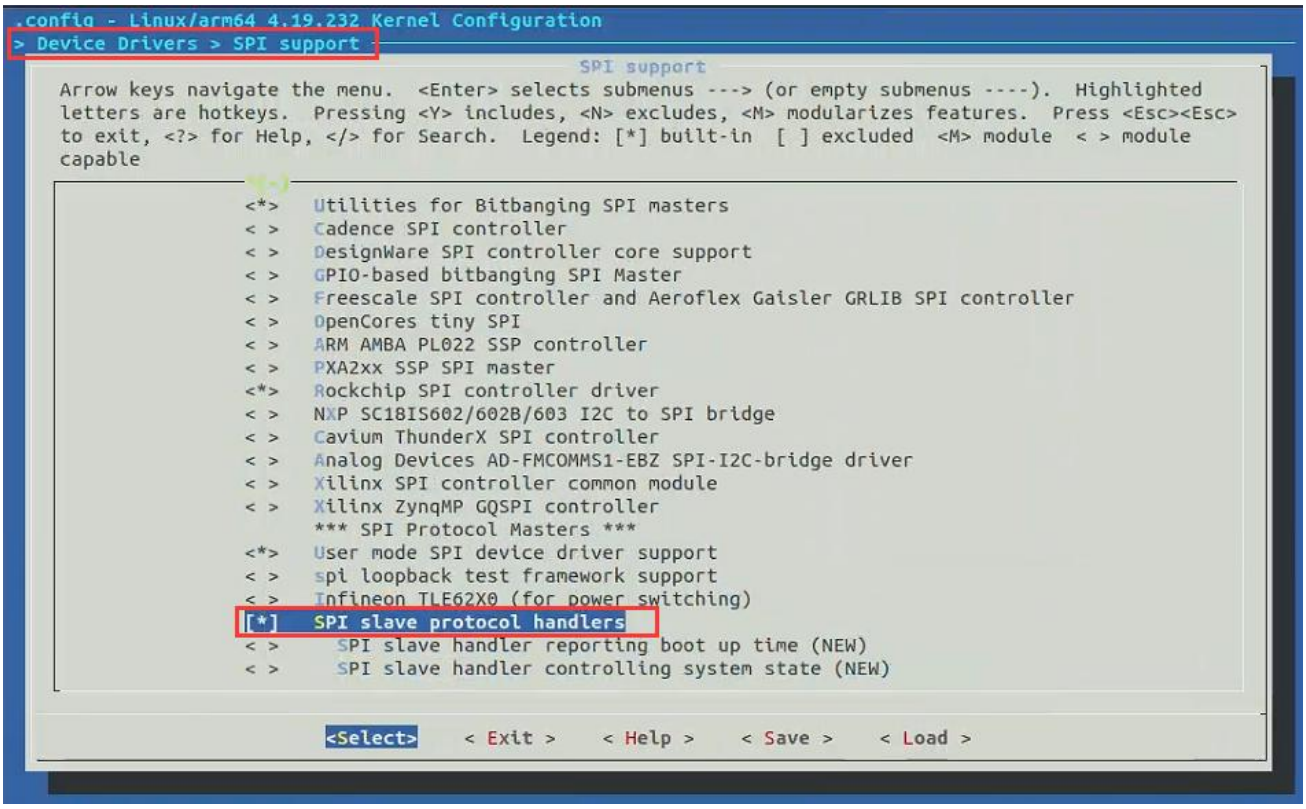
2.2 SPI 代码路径

驱动路径	作用
drivers/spi/spi.c	spi 驱动框架
drivers/spi/spi-rockchip.c rk	spi 各接口实现
drivers/spi/spidev.c	创建 spi 设备节点，用户态使用。
drivers/spi/spi-rockchip-test.c	测试驱动，需要自己手动添加到 Makefile 编译
kernel/tools/spi/spidev_test.c	用户态 spi 测试工具

2.3 RK3568 做 SPI 从设备

如果 RK3568 做从设备，内核配置需要勾选以下配置，如下所示：

```
Device Drivers    --->
    [*]   SPI support    --->
        <*>   SPI slave protocol handlers
```



接下来设备树进行配置，如下所示：

```
&spi1 {
        status = "okay";
        assigned-clocks = <&pmucru CLK_SPI1>;
        assigned-clock-rates = <200000000>;
        dma-names = "tx","rx";
        spi-slave; //使能 slave 模式
          slave { //按照框架要求，SPI slave 子节点的命名需以 "slave" 开始
          compatible ="rockchip,spi_test_bus1_cs0";
          reg = <0>;
          id = <0>;
          //spi-max-frequency = <24000000>; //这不需要配
        };
};
```

注意：

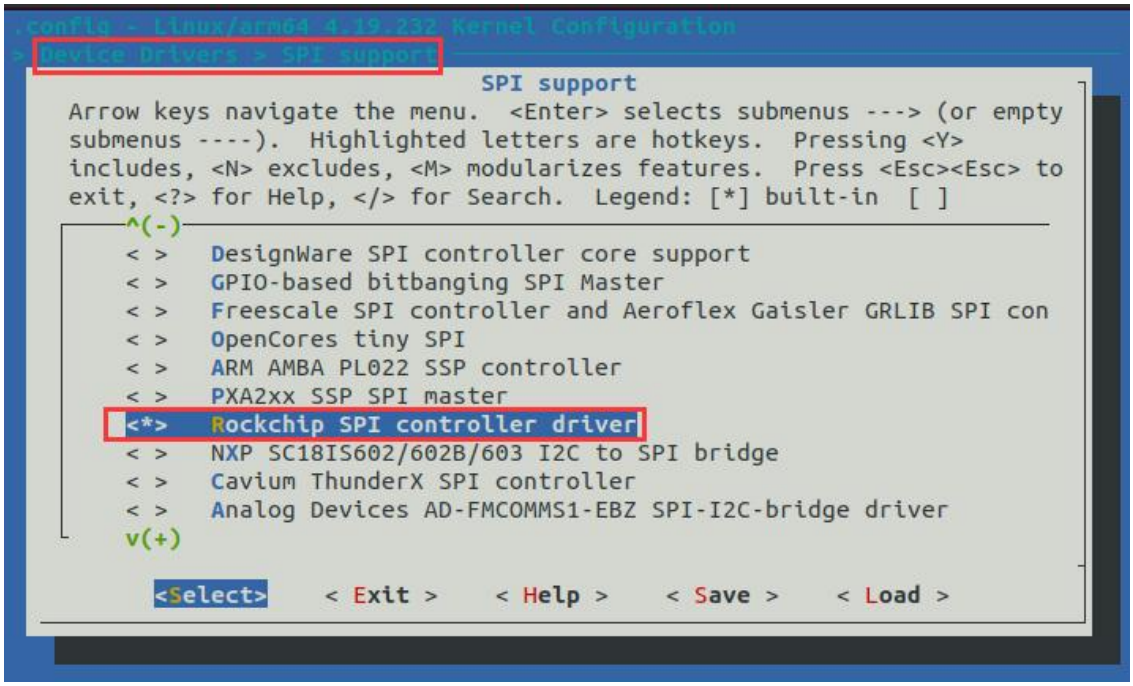
1 spi_clk assigned-clock-rates 必须是 master spi-max-frequency clk 的 6 倍以上，比如 spi_clk assigned-clock-rates = <48000000>，master 给过来的时钟必须小于 8M。

2 实际使用场景可以考虑主从之间增加一个 gpio，主设备输出来通知从设备 transfer ready 来减少 CPU 忙等时间。

2.4 RK3568 做 SPI 主设备

如果 RK3568 做 SPI 主设备，内核配置需要勾选以下配置，默认迅为提供的源码是支持此选项的。

```
Device Drivers    --->
    [*]   SPI support    --->
        <*>   Rockchip SPI controller driver
```



接下来设备树进行配置，比如使用 SPI1，设备树配置如下：

```
&spi1 { //引用 spi 控制器节点
    status = "okay";
    //assigned-clock-rates = <200000000>; //默认不用配置，SPI 设备工作时钟值，io 时钟由工作时钟分频获取
    dma-names = "tx","rx"; //使能 DMA 模式，通讯长度少于 32 字节不建议用，置空赋值去掉使能，如"dma-names;";
    //rx-sample-delay-ns = <10>; //默认不用配置，读采样延时
    spi_test@10 {
        compatible ="rockchip,spi_test_bus1_cs0"; //与驱动对应的名字
        reg = <0>; //片选 0 或者 1
        spi-cpha; //设置 CPHA = 1，不配置则为 0
```

```
            spi-cpol; //设置 CPOL = 1，不配置则为 0
            spi-lsb-first; //IO 先传输 lsb
            spi-max-frequency = <24000000>; //spi clk 输出的时钟频率，不超过 50M
            status = "okay"; //使能设备节点
        };
};
```

spiclk assigned-clock-rates 和 spi-max-frequency 的配置说明：

1、spi-max-frequency 是 SPI 的输出时钟，由 SPI 工作时钟 assigned-clock-rates 内部分频后输出，由于内部至少 2 分频，所以 assigned-clock-rates>=2*spi-max-frequency

2、assigned-clock-rates 不要低于 24M，否则可能有问题

3、假定需要 50MHz 的 SPI IO 速率，可以考虑配置（记住内部分频为偶数分频）spi_clk assigned-clock-rates = <100000000>，spi-max-frequency = <50000000>，即工作时钟 100 MHz（PLL 分频到一个不大于 100MHz 但最接近的值），然后内部二分频最终 IO 接近 50 MHz；

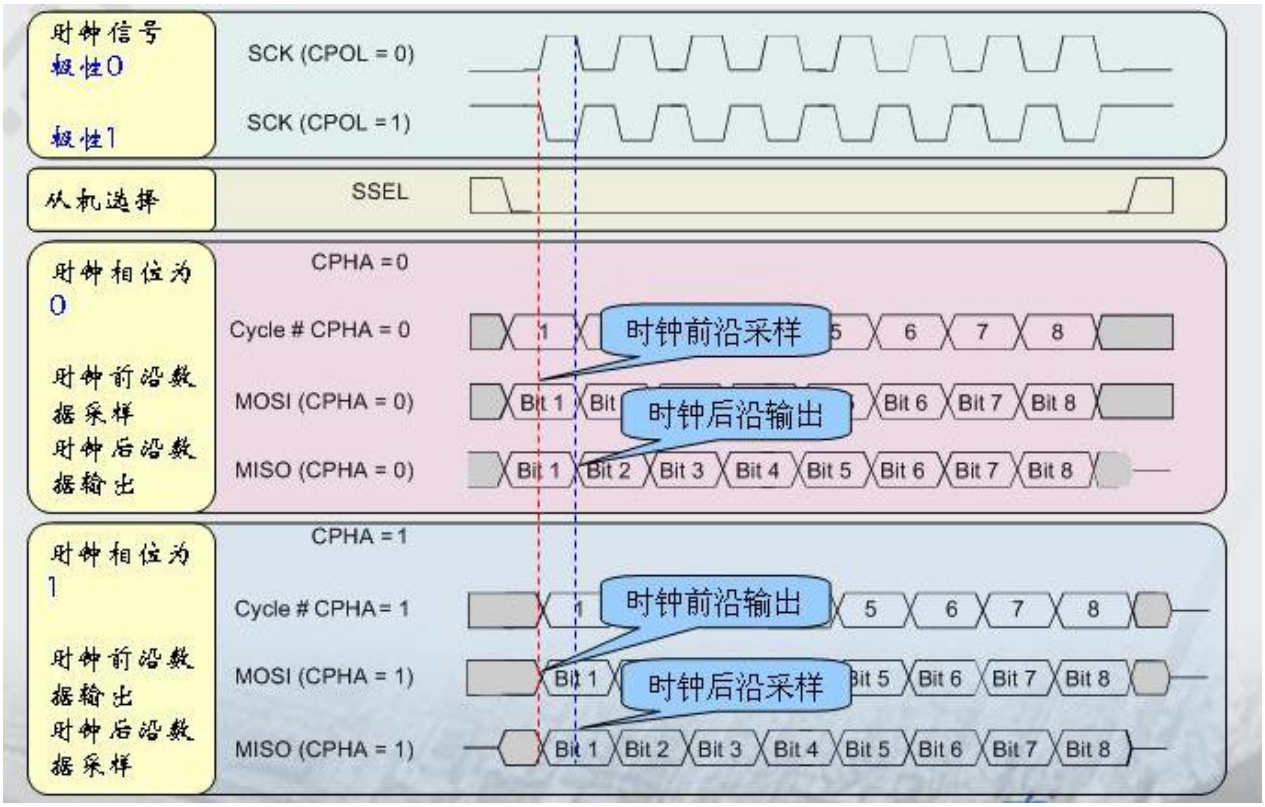
4、Linux 内核用 CPOL 和 CPHA 组合来表示当前 SPI 的四种工作模式：

CPOL：表示时钟信号的初始电平的状态，0 为低电平，1 为高电平

CPHA：表示在哪个时钟沿采样，0 为第一个时钟沿采样，1 为第二个时钟沿采样

CPOL=0，CPHA=0	SPI_MODE_0
CPOL=0，CPHA=1	SPI_MODE_1
CPOL=1，CPHA=0	SPI_MODE_2
CPOL=1，CPHA=1	SPI_MODE_3

SPI 的四种工作模式波形图如下所示：



2.5 SPI 驱动

设备驱动使用 spi 需要调用 spi_register_driver 注册 spi 设备。

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/platform_device.h>
#include <linux/of.h>
#include <linux/spi/spi.h>

static int spi_test_probe(struct spi_device *spi)
{
    int ret;

    if(!spi)
        return -ENOMEM;
    spi->bits_per_word= 8;
    ret= spi_setup(spi);
    if(ret < 0) {
```

```
            dev_err(&spi->dev,"ERR: fail to setup spi\n");
            return-1;
        }
        return ret;
    }

    static int spi_test_remove(struct spi_device *spi)
    {
        printk("%s\n",__func__);
        return 0;
    }

    static const struct of_device_id spi_test_dt_match[]= {
        {.compatible = "rockchip,spi_test_bus1_cs0", },
        {.compatible = "rockchip,spi_test_bus1_cs1", },
        {},
    };

    MODULE_DEVICE_TABLE(of, spi_test_dt_match);

    static struct spi_driver spi_test_driver = {
        .driver = {
            .name = "spi_test",
            .owner = THIS_MODULE,
            .of_match_table = of_match_ptr(spi_test_dt_match),
        },
        .probe = spi_test_probe,
        .remove = spi_test_remove,
    };

    static int __init spi_test_init(void)
    {
        int ret = 0;
        ret = spi_register_driver(&spi_test_driver);
```

```
        return ret;
    }
    module_init(spi_test_init);

    static void __exit spi_test_exit(void)
    {
        return spi_unregister_driver(&spi_test_driver);
    }
    module_exit(spi_test_exit);
```

对于 SPI 读写操作请参考 include/linux/spi/spi.h，以下简单列举：

```
static inline int spi_write(struct spi_device *spi,const void *buf, size_t len)
static inline int spi_read(struct spi_device *spi,void *buf, size_t len)
static inline int spi_write_and_read(structspi_device *spi, const void *tx_buf, void *rx_buf,size_t len)
```

2.6 SPI 回环测试

测试配套资料在“iTOP-3568 开发板\02_【iTOP-RK3568 开发板】开发资料\09_系统外设功能测试\07_外设接口配置配套资料\01_SPI 回环测试配套资料”目录下。

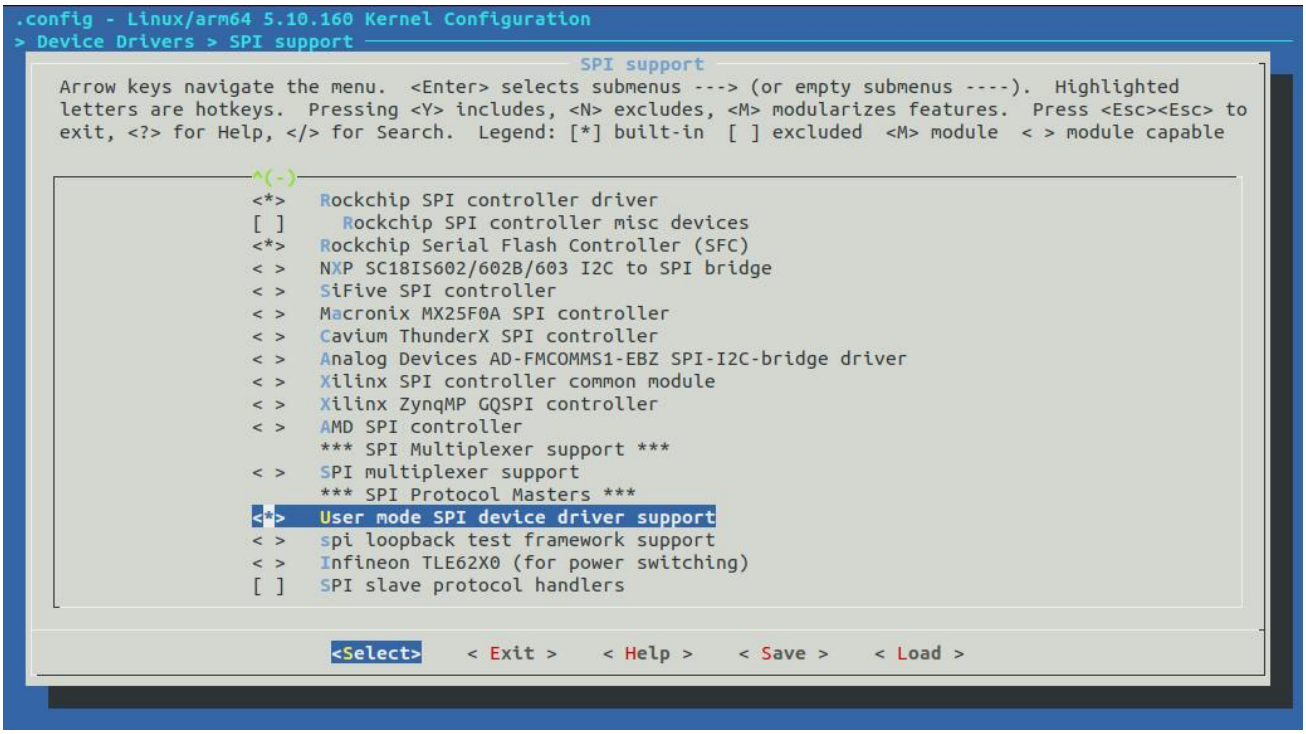
2.6.1 配置驱动

使用以下命令打开 RK3568 内核默认配置界面并勾选以下内容，如下图所示：

```
export ARCH=arm64
make rockchip_linux_defconfig
make menuconfig
```

内核勾选以下配置,默认迅为提供的源码是支持此选项的，如下图所示：

```
Device Drivers    --->
    [*]   SPI support    --->
        <*>   User mode SPI device driver support
```



2.6.2 修改设备树

在/kernel/arch/arm64/boot/dts/rockchip/rk3568-evb1-ddr4-v10.dtsi 末尾添加以下内容：

```
&spi0 {
        status = "okay";
        pinctrl-0 = <&spi0m1_cs0 &spi0m1_pins>;
        pinctrl-1 = <&spi0m1_cs0 &spi0m1_pins_hs>;
        spi_test@00 {
                status = "okay";
                compatible = "rockchip,spidev";
                reg = <0x00>;
                spi-max-frequency = <24000000>;
        };
};
```

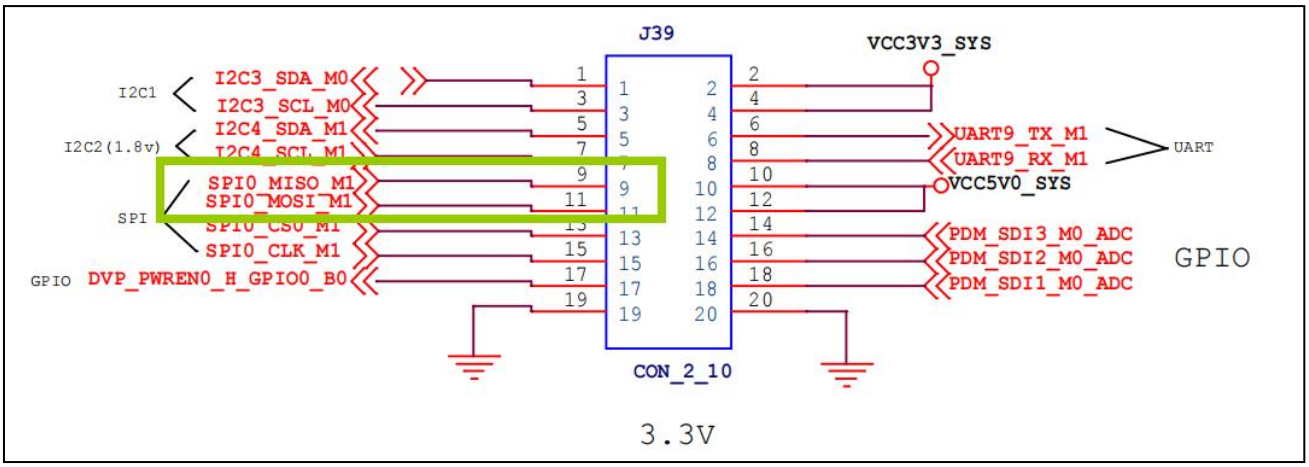
修改完保存退出，单独编译内核生成 boot.img 单独烧写即可。

2.6.3 回环测试

内核设备树都修改后会生成/dev/spidev0.0 这个设备节点，第一个 0 代表 spi 总线 0，第二个 0 代表这个总线上的第 0 个设备。

测试配套资料在“iTOP-3568 开发板\02_【iTOP-RK3568 开发板】开发资料\09_系统外设功能测试\07_外设接口配置配套资料\01_SPI 回环测试配套资料”目录下。

首先将 MOSI 和 MISO 对应这两个 gpio 短接在一起，然后执行如下测试:



使用以下命令进行回环测试，如下图所示：

```
./spi_test /dev/spidev0.0
```

打印信息如下图，表示收发正常。

```
root@rk3568-buildroot:/#
root@rk3568-buildroot:/# ./spi_test /dev/spidev0.0
spi mode: 0
bits per word: 8
max speed: 500000 Hz (500 KHz)

t o p e e t
! t o p e e
t ! t o p e
e t ! t o p
e e t ! t o
p e e t !
root@rk3568-buildroot:/#
root@rk3568-buildroot:/#
```

2.7 常见问题

如果 SPI 无信号，调试方法如下所示：

1 调试前确认驱动有跑起来

2 确保 SPI 四个引脚的 IOMUX 配置无误

3 确认 TX 发送的时候，TX 引脚有正常的波形，CLK 有正常的 CLOCK 信号，CS 信号有拉低

4 如果 clk 频率较高，可以考虑提高驱动强度来改善信号

5 如何简单判断 SPI DMA 是否使能，串口如果没有以下打印，那么 DMA 使能成功

```
[ 0.457137] Failed to request TX DMA channel
[ 0.457237] Failed to request RX DMA channel
```

延迟采样时钟配置方案

对于 SPI IO 速率较高的情形，正常 SPI mode 可能依旧无法匹配外接器件输出延时，RK SPI master read 可能无法采到有效数据，需要启用 SPI rsd 逻辑来延迟采样时钟。

RK SPI rsd(read sample delay)控制逻辑有以下特性：

- 可配置为 0，1，2，3
- 延时单位为 1 spi_clk cycle，即控制器工作时钟

Rx-sample-delay 实际延时为 dts 设定值最接近的 rst 有效值为准，以 spi_clk 200MHz，周期 5ns 为例：rsd 实际可配延迟为 0,5ns,10ns,15ns,rx-sample-delay 设定 12ns，接近有效值 10ns,所以最终为 10ns 延时。

第 3 章 ADC 使用

iTOP-RK3568 开发板上的 AD 接口有两种，分别是温度传感器和逐次逼近 ADC，其中：

TS-ADC（Temperature Sendor）:支持俩通道，时钟频率必须低于 800kHZ

SAR-ADC（Successive Approximation Register）:支持六通道单端 10 位的 SAR-ADC，时钟频率必须小于 13MHZ。

Linux 内核采用工业 I/O 子系统来控制 ADC，该子系统主要为 AD 转换或者 DA 转换的传感器设计。下面以 SAR-ADC 为例子，介绍 ADC 的基本配置方法。

3.1 设备树配置

iTOP-RK3568 开发板 SAR-ADC 的设备树节点在 rk356x_linux/kernel/arch/arm64/boot/dts/rockchip/rk3568.dtsi 文件中定义，如下所示：

```
saradc: saradc@fe720000 {
        compatible = "rockchip,rk3568-saradc", "rockchip,rk3399-saradc";
        reg = <0x0 0xfe720000 0x0 0x100>;
        interrupts = <GIC_SPI 93 IRQ_TYPE_LEVEL_HIGH>;
        #io-channel-cells = <1>;
        clocks = <&cru CLK_SARADC>, <&cru PCLK_SARADC>;
        clock-names = "saradc", "apb_pclk";
        resets = <&cru SRST_P_SARADC>;
        reset-names = "saradc-apb";
        status = "disabled";
};
```

用户首先需要在设备树中添加 ADC 的资源描述，以开发板按键为例：

```
adc_keys: adc-keys {
        compatible = "adc-keys";
        io-channels = <&saradc 0>;
        io-channel-names = "buttons";
        keyup-threshold-microvolt = <1800000>;
        poll-interval = <100>;
        vol-up-key {
                label = "volume up";
```

```
                linux,code = <KEY_VOLUMEUP>;
                press-threshold-microvolt = <1750>;
        };
        vol-down-key {
                label = "volume down";
                linux,code = <KEY_VOLUMEDOWN>;
                press-threshold-microvolt = <297500>;
        };
        menu-key {
                label = "menu";
                linux,code = <KEY_MENU>;
                press-threshold-microvolt = <980000>;
        };
        back-key {
                label = "back";
                linux,code = <KEY_BACK>;
                press-threshold-microvolt = <1305500>;
        };
};
```

设备树解析：

- io-channels --> 引用使用哪一路 ADC
- io-channels 属性 为选择的通道号
- io-channel-names 属性 表示 为申请的通道起一个别名。
- keyup-threshold-microvolt 属性 表示按键抬起，saradc 通道 1 的电压（单位微伏）。
- poll-interval 表示获取 ADC 值的轮询间隔配置
- press-threshold-microvolt 属性 表示按键按下，saradc 通道 1 的电压。
- vol-up-key, vol-down-key, menu-key, back-key 在硬件连接上，
- linux,code 属性 为 按键上报的键值，键值对应的动作 为“音量+”。
- label 表示 为按键起一个别名。

3.2 在驱动文件中匹配 DTS 节点

驱动：kernel/drivers/input/keyboard/adc-keys.c
首先在驱动文件中定义 of_device_id 结构体数组：

```
static const struct of_device_id adc_keys_of_match[] = {
```

```
        { .compatible = "adc-keys", },
        { }
};
```

然后将该结构体数组填充到要使用 ADC 的 platform_driver 中。

```
static struct platform_driver __refdata adc_keys_driver = {
        .driver = {
                .name = "adc_keys",
                .of_match_table = of_match_ptr(adc_keys_of_match),
        },
        .probe = adc_keys_probe,
};
```

接着在 adc_keys_probe 中对设备树所添加的资源进行解析

```
 static int adc_keys_probe(struct platform_device *pdev)
{
        struct device *dev = &pdev->dev;
        struct adc_keys_state *st;
        struct input_polled_dev *poll_dev;
        struct input_dev *input;
        enum iio_chan_type type;
        int i, value;
        int error;

        st = devm_kzalloc(dev, sizeof(*st), GFP_KERNEL);
        if (!st)
                return -ENOMEM;

        st->channel = devm_iio_channel_get(dev, "buttons");
        if (IS_ERR(st->channel))
                return PTR_ERR(st->channel);

        if (!st->channel->indio_dev)
                return -ENXIO;

        error = iio_get_channel_type(st->channel, &type);
        if (error < 0)
```

```
                return error;

        if (type != IIO_VOLTAGE) {
                dev_err(dev, "Incompatible channel type %d\n", type);
                return -EINVAL;
        }

        if (device_property_read_u32(dev, "keyup-threshold-microvolt",
                                     &st->keyup_voltage)) {
                dev_err(dev, "Invalid or missing keyup voltage\n");
                return -EINVAL;
        }
        st->keyup_voltage /= 1000;

        error = adc_keys_load_keymap(dev, st);
        if (error)
                return error;

        poll_dev = devm_input_allocate_polled_device(dev);
        if (!poll_dev) {
                dev_err(dev, "failed to allocate input device\n");
                return -ENOMEM;
        }

        if (!device_property_read_u32(dev, "poll-interval", &value))
                poll_dev->poll_interval = value;

        poll_dev->poll = adc_keys_poll;
        poll_dev->private = st;

        input = poll_dev->input;

        input->name = pdev->name;
        input->phys = "adc-keys/input0";
```

```
        input->id.bustype = BUS_HOST;
        input->id.vendor = 0x0001;
        input->id.product = 0x0001;
        input->id.version = 0x0100;

        __set_bit(EV_KEY, input->evbit);
        for (i = 0; i < st->num_keys; i++)
                __set_bit(st->map[i].keycode, input->keybit);

        if (device_property_read_bool(dev, "autorepeat"))
                __set_bit(EV_REP, input->evbit);

        error = input_register_polled_device(poll_dev);
        if (error) {
                dev_err(dev, "Unable to register input device: %d\n", error);
                return error;
        }

        return 0;
}
```

3.3 驱动说明

- 关键的上报按键事件的代码

```
static void adc_keys_poll(struct input_polled_dev *dev)
{
    struct adc_keys_state *st = dev->private;
    int i, value, ret;
    u32 diff, closest = 0xffffffff;
    int keycode = 0;

    ret = iio_read_channel_processed(st->channel, &value);//读取 adc 通道值
    if (unlikely(ret < 0)) {
        /* Forcibly release key if any was pressed */
```

```
            // 如果未按下任何键，则强制释放键
            value = st->keyup_voltage;
        } else {
            for (i = 0; i < st->num_keys; i++) {
                //st->map[i].voltage = "press-threshold-microvolt" = 设备树设定的按下时 adc 值
                diff = abs(st->map[i].voltage - value);
                if (diff < closest) {//找出最接近的键值
                    closest = diff;
                    keycode = st->map[i].keycode;
                }
            }
        }
    //如果 adc 的值与开路电压的 adc 的误差比上面的最小误差都小，那么判定为未按下按键
        if (abs(st->keyup_voltage - value) < closest)
            keycode = 0;
        //如果上一次有键值，并且这次的键值和上次不一样，就说明上一次的按键已经松开
        //并且这次检测，又有新的按键被按下，就先发送上次键值的松开事件
        if (st->last_key && st->last_key != keycode)
            input_report_key(dev->input, st->last_key, 0);
         //再发送新的键值的按下事件
        if (keycode)
            input_report_key(dev->input, keycode, 1);

        input_sync(dev->input);
        st->last_key = keycode;
    }
```

- 计算采集到的电压

使用标准电压将 AD 转换的值转换为用户所需要的电压值。其计算公式是

```
Vref/(2^n-1) = Vresult /raw
```

Vref 为标准电压，n 为 AD 转换的位数，Vresult 为用户所需要的采集电压，raw 为 AD 采集的原始数据。

例如：标准电压是 1.8V，AD 采集位数是 10 位，AD 采集到的原始数据是 568，那么 Vresult = (1800mv * 568) / 1023;

3.4 调试方法

具体测试可参考群文件“02_开发板启动和测试/02_【北京迅为】itop-3568 开发板快速测试手册【底板 v1.7 版】”手册按键测试章节。

获取所有 ADC 值，输入以下命令：

```
cat /sys/bus/iio/devices/iio\:device0/in_voltage*_raw
```

```
/iio\:device0/in_voltage*_raws/iio/devices/iio:device0# cat /sys/bus/iio/devices/
1023
1023
1010
1022
1007
1012
1004
1001
```

电压采集计算：

标准电压为 1.8V，AD 采集位数为 10 位，AD 采集到的原始数据为 1023，Vresult = (1800mv * 1023) / 1023=1.8v

如果是自己用 adc 那么设备树开启对应通道即可：

```
adc_demo: adc-demo {
        status = "okay";
        compatible = "adc-keys";
        io-channels = <&saradc x>;
};
```

第 4 章 LED 使用

iTOP-RK3568 开发板上有一个用户 LED 灯，如下所示：

LED	Pin name	Pin number
LED9	GPIO0_B7	15

可以通过 LED 设备子系统或者直接操作 GPIO 控制该 LED 灯。

4.1 设备树编写

设备树节点如下所示，在 rk356x_linux/kernel/arch/arm64/boot/dts/rockchip/rk3568-evb.dtsi 文件中。

```
leds: leds {
    compatible = "gpio-leds";
    work_led: work {
        lable = "user1";
        gpios = <&gpio0 RK_PB7 GPIO_ACTIVE_HIGH>;
        linux,default-trigger = "heartbeat";
        default-state = "on";
    };
```

上述描述中，lable 是可选属性，表示 led 的名字。default-state 是可选属性，表示默认 led 打开“on”,还可以设置为 off。gpios 表示 led 灯的引脚，可以根据原理图进行修改。

上述配置为当开发板供电时，LED 灯常亮，用户可以关闭该功能，只需要将 leds 节点属性 default-state = "on"改成”off”, linux,default-trigger 改成"none";

```
leds: leds {
    compatible = "gpio-leds";
    work_led: work {
        lable = "user1";
        gpios = <&gpio0 RK_PB7 GPIO_ACTIVE_HIGH>;
        linux,default-trigger = "none";
        default-state = "off";
    };
};
```

4.2 LED 子系统

4.2.1 用户态

标准的 Linux 专门为 LED 设备定义了 LED 子系统。在 iTOP-RK3568 开发板中的 LED 是以设备的方式被定义的。在用户态，可以直接访问节点/sys/class/leds/XXX 下的文件操作 LED。

在/sys/class/leds/XXX 路径下会有 brightness,max_brightness,trigger ,根据 trigger 的不同，还可能有 delay_off,delay_on,invert 等。

- brightness 用于设置 LED 亮度，范围是 0~255
- max_brightness 用于显示 LED 的最大亮度访问，一般值为 255。
- trigger 用于设置 LED 的触发模式，通常可选的有 none,nand-disk,mmc0,cpu0,heartbeat,timer,default-on,oneshot,backlight,gpio;
- delay_off、delay_on 用于 trigger 为 timer 等模式时，LED 亮灭的时间，单位为毫秒。

4.2.2 内核驱动

LED 子系统的驱动全部都在 drivers/leds 下面，如下所示：

```
topeet@ubuntu:~/Linux/rk356x_linux/kernel/drivers/leds$ ls
built-in.a          leds-asic3.c        leds-dac124s085.c   leds-lm3530.c       leds-lp55xx-common.c  leds-mt6323.c       leds-pwm.c            leds-wm8350.c
Kconfig             leds-bcm6328.c      leds-fsg.c          leds-lm3533.c       leds-lp55xx-common.h  leds-multi.h        leds-rb532.c          leds-wrap.c
led-class.c         leds-bcm6358.c      leds-gpio.c         leds-lm355x.c       leds-lp8501.c         leds-net48xx.c      leds-regulator.c      led-triggers.c
led-class-flash.c   leds-bd2802.c       leds-gpio.o         leds-lm3601x.c      leds-lp8788.c         leds-netxbig.c      leds-rgb13h.c         led-triggers.o
led-class.o         leds-blinkm.c       leds-gpio-register.c leds-lm3642.c      leds-lp8860.c         leds-nic78bx.c      leds-s3c24xx.c        Makefile
led-core.c          leds-clevo-mail.c   leds.h              leds-lm3692x.c      leds-lt3593.c         leds-ns2.c          leds-sc27xx-bltc.c    modules.builtin
led-core.o          leds-cobalt-qube.c  leds-hp6xx.c        leds-locomo.c       leds-max77693.c       leds-ot200.c        leds-ss4200.c         modules.order
leds-88pm860x.c     leds-cobalt-raq.c   leds-ipaq-micro.c   leds-lp3944.c       leds-max8997.c        leds-pca9532.c      leds-sunfire.c        trigger
leds-aat1290.c      leds-cpcap.c        leds-is31fl319x.c   leds-lp3952.c       leds-mc13783.c        leds-pca955x.c      leds-syscon.c         uleds.c
leds-adp5520.c      leds-cr0014114.c    leds-is31fl32xx.c   leds-lp5521.c       leds-menf21bmc.c      leds-pca963x.c      leds-tca6507.c
leds-apu.c          leds-da903x.c       leds-is31fl32xx.o   leds-lp5523.c       leds-mlxcpld.c        leds-pm8058.c       leds-tlc591xx.c
leds-as3645a.c      leds-da9052.c       leds-ktd2692.c      leds-lp5562.c       leds-mlxreg.c         leds-powernv.c      leds-wm831x-status.c
topeet@ubuntu:~/Linux/rk356x_linux/kernel/drivers/leds$
```

核心文件是 led-class.c、led-core.c、led-triggers.c，可以选择的触发方式有：

```
obj-$(CONFIG_LEDS_TRIGGER_TIMER)        += ledtrig-timer.o        //定时触发
obj-$(CONFIG_LEDS_TRIGGER_ONESHOT)      += ledtrig-oneshot.o      //单次触发
obj-$(CONFIG_LEDS_TRIGGER_IDE_DISK)     += ledtrig-ide-disk.o     //硬盘触发
obj-$(CONFIG_LEDS_TRIGGER_HEARTBEAT)    += ledtrig-heartbeat.o    //心跳触发
obj-$(CONFIG_LEDS_TRIGGER_BACKLIGHT)    += ledtrig-backlight.o    //背光设置
obj-$(CONFIG_LEDS_TRIGGER_GPIO)         += ledtrig-gpio.o         //GPIO 触发
obj-$(CONFIG_LEDS_TRIGGER_CPU)          += ledtrig-cpu.o          //CPU 触发
obj-$(CONFIG_LEDS_TRIGGER_DEFAULT_ON)   += ledtrig-default-on.o //默认开
obj-$(CONFIG_LEDS_TRIGGER_TRANSIENT)    += ledtrig-transient.o
obj-$(CONFIG_LEDS_TRIGGER_CAMERA)       += ledtrig-camera.o
```

leds-gpio.c 和 leds-xxx.c 对应具体的设备，比如某个 led 接在了 GPIO 上，理论上需要我们

编写 leds-gpio.c 驱动文件，但是 LED 子系统已经帮我们做好了，我们只需要在设备树中添加相应的设备信息即可。

接下来我们对主要文件的内容进行分析。

led-core.c 文件抽象出 LED 操作逻辑，封装成函数导出以供其他文件使用。

led_init_core():核心初始化

led_blink_set():设置 led 闪烁时间

led_blink_set_oneshot():闪烁一次

led_stop_software_blink():led 停止闪烁

led_set_brightness():设置 led 的亮度

led_set_brightness_nopm():如果可以休眠，设置 led 的亮度保证休眠可用

led_set_brightness_nosleep():如果没有休眠，设置 led 的亮度

led_set_brightness_sync():设置 led 亮度同步

led_update_brightness():更新亮度

led_sysfs_disable：用户态关闭

led_sysfs_enable: 用户态打开

leds_list:leds 链表

leds_list_lock:leds 链表所

led-class.c 文件维护 LED 子系统的所有 LED 设备，为 LED 设备提供以下函数：

注册操作函数：

led_classdev_register()/of_led_classdev_register()/devm_of_led_classdev_register()/devm_led_classdev_register()

注销操作函数：

led_classdev_unregister()/devm_led_classdev_unregister();

电源管理的休眠和恢复操作函数：

led_classdev_suspend(),led_classdev_resume();

同时提供基本的用户态操作接口：brightness、max_brightness 等。

led-triggers.c 文件维护 LED 子系统的所有触发器为触发器提供以下函数：

注册操作函数：

led_trigger_register(),devm_led_trigger_register(),led_trigger_register_simple();

注销操作函数：led_trigger_unregister(),led_trigger_unregister_simple();以及其他触发器相关的操作函数。

ledtrig-timer.c ledtrig-xxx.c

以 ledtrig-timer.c 为例的触发器，入口函数调用 led_trigger_register()注册触发器，注册时候传入 led_trigger 结构体，里面有 activate 和 deactivate 成员函数指针，这里的作用是生成 delay_on，

delay_off 文件。同时还提供 delay_on 和 delay_off 的用户态操作接口。卸载时，使用 led_trigger_unregister()注销触发器。

leds-gpio.c leds-xxx.c

以 leds-gpio.c 为例的 LED 设备，在通过设备树或者其他途径匹配到设备信息后，将调用 probe()函数，然后再根据设备信息设置 led_classdev，最后调用 devm_of_led_classdev_register()注册 LED 设备。

4.3 LED 驱动

rk356x_linux/kernel/drivers/leds/leds-gpio.c 文件中注册平台设备驱动。如下所示：

```
static const struct of_device_id of_gpio_leds_match[] = {
    { .compatible = "gpio-leds", },
    {},
};

MODULE_DEVICE_TABLE(of, of_gpio_leds_match);
.......
static struct platform_driver gpio_led_driver = {
    .probe          = gpio_led_probe,
    .shutdown       = gpio_led_shutdown,
    .driver         = {
        .name       = "leds-gpio",
        .of_match_table = of_gpio_leds_match,
    },
};

module_platform_driver(gpio_led_driver);
```

- of_match_table 表用于驱动和设备匹配，这里将匹配主设备 leds 节点。
- MODULE_DEVICE_TABLE(of, of_gpio_leds_match);这个宏会生成一个符号表，把该设备加入到设备队列中，这种方式支持使用平台设备匹配驱动。
- platform_driver 定义了平台驱动结构体，填充 of_match_table 段，设备树的方式匹配驱动。
- 使用 module_platform_driver 宏注册平台驱动，该 led 驱动是一个 platform_driver 的对象。

平台设备驱动的 probe 函数如下所示：

```
static int gpio_led_probe(struct platform_device *pdev)
{
```

```
        /*获取 platform_data,使用设备树方式描述硬件，pdata 为 NULL*/
    struct gpio_led_platform_data *pdata = dev_get_platdata(&pdev->dev);
        struct gpio_leds_priv *priv;
        int i, ret = 0;

        if (pdata && pdata->num_leds) {    /*获取到 pdata,使用设备 platform_device 方式处理硬件资源*/
            priv = devm_kzalloc(&pdev->dev,
                    sizeof_gpio_leds_priv(pdata->num_leds),
                        GFP_KERNEL);
            if (!priv)
                return -ENOMEM;

            priv->num_leds = pdata->num_leds;
            for (i = 0; i < priv->num_leds; i++) {
                ret = create_gpio_led(&pdata->leds[i], &priv->leds[i],
                                &pdev->dev, NULL,
                                pdata->gpio_blink_set);
                if (ret < 0)
                    return ret;
            }
        } else {     /*设备树方式*/
            priv = gpio_leds_create(pdev);    /*获取设备节点的资源，设置相关属性*/
            if (IS_ERR(priv))
                return PTR_ERR(priv);
        }

        platform_set_drvdata(pdev, priv);

        return 0;
    }
```

gpio_leds_create 函数如下所示：

```
    static struct gpio_leds_priv *gpio_leds_create(struct platform_device *pdev)
    {
```

```
struct device *dev = &pdev->dev;
struct fwnode_handle *child;
struct gpio_leds_priv *priv;
int count, ret;

count = device_get_child_node_count(dev);
if (!count)
	return ERR_PTR(-ENODEV);

priv = devm_kzalloc(dev, sizeof_gpio_leds_priv(count), GFP_KERNEL);
if (!priv)
	return ERR_PTR(-ENOMEM);

device_for_each_child_node(dev, child) {
	struct gpio_led_data *led_dat = &priv->leds[priv->num_leds];
	struct gpio_led led = {};
	const char *state = NULL;
	struct device_node *np = to_of_node(child);

	ret = fwnode_property_read_string(child, "label", &led.name);
	if (ret && IS_ENABLED(CONFIG_OF) && np)
		led.name = np->name;
	if (!led.name) {
		fwnode_handle_put(child);
		return ERR_PTR(-EINVAL);
	}

	led.gpiod = devm_fwnode_get_gpiod_from_child(dev, NULL, child,
						GPIOD_ASIS,
						led.name);
	if (IS_ERR(led.gpiod)) {
		fwnode_handle_put(child);
		return ERR_CAST(led.gpiod);
	}
```

```
        fwnode_property_read_string(child, "linux,default-trigger",
                                    &led.default_trigger);

        if (!fwnode_property_read_string(child, "default-state",
                                    &state)) {
            if (!strcmp(state, "keep"))
                led.default_state = LEDS_GPIO_DEFSTATE_KEEP;
            else if (!strcmp(state, "on"))
                led.default_state = LEDS_GPIO_DEFSTATE_ON;
            else
                led.default_state = LEDS_GPIO_DEFSTATE_OFF;
        }

        if (fwnode_property_present(child, "retain-state-suspended"))
            led.retain_state_suspended = 1;
        if (fwnode_property_present(child, "retain-state-shutdown"))
            led.retain_state_shutdown = 1;
        if (fwnode_property_present(child, "panic-indicator"))
            led.panic_indicator = 1;

        ret = create_gpio_led(&led, led_dat, dev, np, NULL);
        if (ret < 0) {
            fwnode_handle_put(child);
            return ERR_PTR(ret);
        }
        led_dat->cdev.dev->of_node = np;
        priv->num_leds++;
    }

    return priv;
}
```

create_gpio_led 函数如下所示：

```
static int create_gpio_led(const struct gpio_led *template,
```

```
        struct gpio_led_data *led_dat, struct device *parent,
        struct device_node *np, gpio_blink_set_t blink_set)
{
        int ret, state;

        led_dat->gpiod = template->gpiod;
        if (!led_dat->gpiod) {
                /*
                 * This is the legacy code path for platform code that
                 * still uses GPIO numbers. Ultimately we would like to get
                 * rid of this block completely.
                 */
                unsigned long flags = GPIOF_OUT_INIT_LOW;

                /* skip leds that aren't available */
                if (!gpio_is_valid(template->gpio)) {
                        dev_info(parent, "Skipping unavailable LED gpio %d (%s)\n",
                                 template->gpio, template->name);
                        return 0;
                }

                if (template->active_low)
                        flags |= GPIOF_ACTIVE_LOW;

                ret = devm_gpio_request_one(parent, template->gpio, flags,
                                            template->name);
                if (ret < 0)
                        return ret;

                led_dat->gpiod = gpio_to_desc(template->gpio);
                if (!led_dat->gpiod)
                        return -EINVAL;
        }
```

```
	led_dat->cdev.name = template->name;
	led_dat->cdev.default_trigger = template->default_trigger;
	led_dat->can_sleep = gpiod_cansleep(led_dat->gpiod);
	if (!led_dat->can_sleep)
		led_dat->cdev.brightness_set = gpio_led_set;
	else
		led_dat->cdev.brightness_set_blocking = gpio_led_set_blocking;
	led_dat->blinking = 0;
	if (blink_set) {
		led_dat->platform_gpio_blink_set = blink_set;
		led_dat->cdev.blink_set = gpio_blink_set;
	}
	if (template->default_state == LEDS_GPIO_DEFSTATE_KEEP) {
		state = gpiod_get_value_cansleep(led_dat->gpiod);
		if (state < 0)
			return state;
	} else {
		state = (template->default_state == LEDS_GPIO_DEFSTATE_ON);
	}
	led_dat->cdev.brightness = state ? LED_FULL : LED_OFF;
	if (!template->retain_state_suspended)
		led_dat->cdev.flags |= LED_CORE_SUSPENDRESUME;
	if (template->panic_indicator)
		led_dat->cdev.flags |= LED_PANIC_INDICATOR;
	if (template->retain_state_shutdown)
		led_dat->cdev.flags |= LED_RETAIN_AT_SHUTDOWN;

	ret = gpiod_direction_output(led_dat->gpiod, state);
	if (ret < 0)
		return ret;

	return devm_of_led_classdev_register(parent, np, &led_dat->cdev);
}
```

devm_of_led_classdev_register 位于 led-class.c 函数中

```
/**
 * devm_of_led_classdev_register - resource managed led_classdev_register()
 *
 * @parent: parent of LED device
 * @led_cdev: the led_classdev structure for this device.
 */
int devm_of_led_classdev_register(struct device *parent,
                                  struct device_node *np,
                                  struct led_classdev *led_cdev)
{
        struct led_classdev **dr;
        int rc;

        dr = devres_alloc(devm_led_classdev_release, sizeof(*dr), GFP_KERNEL);
        if (!dr)
                return -ENOMEM;

        rc = of_led_classdev_register(parent, np, led_cdev);
        if (rc) {
                devres_free(dr);
                return rc;
        }

        *dr = led_cdev;
        devres_add(parent, dr);

        return 0;
}
EXPORT_SYMBOL_GPL(devm_of_led_classdev_register);
```

4.4 使用方法

重启加载设备树之后，驱动已经编译进内核，会在/sys/class/leds/目录下多 work 目录，进入此目录下，如下所示：

```
root@rk3568-buildroot:/sys/class/leds/work#
root@rk3568-buildroot:/sys/class/leds/work#
root@rk3568-buildroot:/sys/class/leds/work# ls
brightness  device  max_brightness  power  subsystem  trigger  uevent
root@rk3568-buildroot:/sys/class/leds/work#
root@rk3568-buildroot:/sys/class/leds/work#
```

可以通过 echo 0 > brightness 来关闭 LED 灯。同时使用 cat trigger 可以显示支持的触发模式，以及当前的触发方式。

输入以下命令可以控制 led9 的亮灭。

```
echo 0 > brightness    //灯灭
echo 1 > brightness    //灯亮
```

第 5 章 UART 使用

iTOP-RK3568 开发板上支持 UART，RS485 接口。

5.1 设备树配置

串口节点配置如下：

```
&uart4 {    //对应节点/dev/ttyS4
            status = "okay";
            pinctrl-names = "default";
            pinctrl-0 = <&uart4m1_xfer>;
};
&uart9 {    //对应节点/dev/ttyS9
            status = "okay";
            pinctrl-names = "default";
            pinctrl-0 = <&uart9m1_xfer>;
};
```

rs485 节点配置如下：

```
&uart7 {     //对应节点/dev/ttyS7
            status ="okay";
            pinctrl-name = "default";
            pinctrl-0 = <&uart7m1_xfer>;
};
```

5.2 测试

添加设备树之后，重新烧写内核镜像，串口节点如下所示：

```
root@rk3568-buildroot:/# ls /dev/ttyS*
/dev/ttyS4  /dev/ttyS7  /dev/ttyS8  /dev/ttyS9
root@rk3568-buildroot:/#
```

具体测试可看下群文件“02_开发板启动和测试/02_【北京迅为】itop-3568 开发板快速测试手册【底板 v1.7 版】”手册串口测试章节。

5.3 调试串口配置为普通串口

在 iTOP-RK3568 开发板上，UART2 默认为 debug 串口。以下步骤可以将调试串口改为普

通。

1 修改内核源码 kernel/arch/arm64/configs/rockchip_linux_defconfig，去掉 CONFIG_SERIAL_8250_CONSOLE 的配置。

```
删掉：CONFIG_SERIAL_8250_CONSOLE=y
增加：# CONFIG_SERIAL_8250_CONSOLE is not set
```

2 修改 rk356x_linux/kernel/arch/arm64/boot/dts/rockchip/rk3568-linux.dtsi 文件

```
fiq-debugger {
    compatible = "rockchip,fiq-debugger";
    rockchip,serial-id = <2>;
    rockchip,wake-irq = <0>;
    /* If enable uart uses irq instead of fiq */
    rockchip,irq-mode-enable = <1>;
    rockchip,baudrate = <115200>;  /* Only 115200 and 1500000 */
    interrupts = <GIC_SPI 252 IRQ_TYPE_LEVEL_LOW>;
    pinctrl-names = "default";
    pinctrl-0 = <&uart2m0_xfer>;
    status = "okay";  改成disabled
};
```

3 打开 uart2 节点，修改 kernel/arch/arm64/boot/dts/rockchip/rk3568-evb1-ddr4-v10-linux.dts 文件，添加如下节点

```
&uart2 {
        status = "okay";
        pinctrl-names = "default";
        pinctrl-0 = <&uart2m0_xfer>;
};
```

4 重新编译内核并烧录生效后，会生成/dev/ttyS2 节点。

5.4 串口回环测试

测试配套资料在“iTOP-3568 开发板\02_【iTOP-RK3568 开发板】开发资料\09_系统外设功能测试\07_外设接口配置配套资料\02_串口回环测试配套资料”目录下。

以串口 4 为例进行回环测试，将串口的 2 号引脚（Rx）和 3 号引脚（Tx）短接，使用以下命令进行测试，如果在控制台看到有打印接收字节总数，就表明发送和接收成功，如下图所示：

```
./uart_write_read /dev/ttyS4
```

```
[root@RK356X:/]#
[root@RK356X:/]# ./uart_write_read /dev/ttyS4

 uart_write_read_test start
TestDemon input char 'z',TestDemon exit!
open /dev/ttyS4 is success
get data count = 1!
get data count = 2!
get data count = 3!
get data count = 4!
get data count = 5!
get data count = 6!
get data count = 7!
get data count = 8!
get data count = 9!
get data count = 10!
get data count = 11!
get data count = 12!
get data count = 13!
get data count = 14!
get data count = 15!
get data count = 16!
get data count = 17!
get data count = 18!
get data count = 19!
get data count = 20!
get data count = 21!
get data count = 22!
get data count = 23!
get data count = 24!
get data count = 25!
```

第 6 章 IR 使用

iTOP-RK3568 开发板使用红外收发传感器 IR 实现遥控功能，配置步骤分为俩步：

- 步骤一：修改内核设备树。
- 步骤二：修改键值映射

6.1 修改内核空间

在 Linux 内核中，IR 驱动只支持 NEC 编码格式。设备树中 IR 配置如下：注：rockchip,key_table 为第一列键值，第二列为要响应的按键码。

```
&pwm3 {
    status = "okay";
    interrupts = <GIC_SPI 82 IRQ_TYPE_LEVEL_HIGH>;
    compatible = "rockchip,remotectl-pwm";
    remote_pwm_id = <3>;
    handle_cpu_id = <1>;
    remote_support_psci = <0>;
    pinctrl-names = "default";
    pinctrl-0 = <&pwm3_pins>;

    ir_key1 {
        rockchip,usercode = <0xdd22>;
        rockchip,key_table =
            <0x66   KEY_LEFT>,
            <0x3e   KEY_RIGHT>,
            <0x35   KEY_UP>,
            <0x2d   KEY_DOWN>,
            <0x6a   KEY_BACK>,
            <0x31    KEY_ENTER>,
            <0x0c   KEY_VOLUMEUP>,
            <0x0b   KEY_VOLUMEDOWN>,
            <0x7d   KEY_MENU>,
            <0xbe   KEY_1>,
```

```
                <0xba   KEY_2>,
                <0xb2   KEY_3>,
                <0xbd   KEY_4>,
                <0xf9   KEY_5>,
                <0xb1   KEY_6>,
                <0xfc   KEY_7>,
                <0xf8   KEY_8>,
                <0xb0   KEY_9>,
                <0xb6   KEY_0>;
        };


    };
```

在 rockchip_pwm_remotectl.c 驱动中的 remotectl_do_something 函数获取用户码和键值。

```
        case RMC_USERCODE: {
                        if ((RK_PWM_TIME_BIT1_MIN < ddata->period) &&
                            (ddata->period < RK_PWM_TIME_BIT1_MAX))
                                ddata->scandata |= (0x01 << ddata->count);
                        ddata->count++;
                        if (ddata->count == 0x10) {
                                DBG_CODE("USERCODE=0x%x\n", ddata->scandata);
                                if (remotectl_keybd_num_lookup(ddata)) {
                                        ddata->state = RMC_GETDATA;
                                        ddata->scandata = 0;
                                        ddata->count = 0;
                                } else {
                                        if (rk_remote_print_code){
                                                ddata->state = RMC_GETDATA;
                                                ddata->scandata = 0;
                                                ddata->count = 0;
                                        } else
                                                ddata->state = RMC_PRELOAD;
                                }
                        }
                }
```

在 3568Linux 源码中已经存在 IR 驱动 rockchip_pwm_remotectl.c，所以我们需要做的是将 IR 驱动编译进内核，默认配置文件中打开以下选项。

```
CONFIG_ROCKCHIP_REMOTECTL=y
CONFIG_ROCKCHIP_REMOTECTL_PWM=y
```

6.2 用户空间测试

可以使用 DBG_CODE()函数打印用户码。在用户空间，iTOP-RK3568 可以使用以下命令使能 DBG_CODE 打印

```
echo 1 > /sys/module/rockchip_pwm_remotectl/parameters/code_print
```

第 7 章 Ethernet 使用

本章节带大家快速了解有线网卡的配置和使用。

7.1 设备树配置

kernel/arch/arm64/boot/dts/rockchip/rk3568-evb1-ddr4-v10.dtsi 设备树中默认网卡配置如下所示：

```
gmac0 {    //eth0
    phy-mode = "rgmii";
    clock_in_out = "input";
    snps,reset-gpio = <&gpio3 RK_PB7 GPIO_ACTIVE_LOW>;
    snps,reset-active-low;
    /* Reset time is 20ms, 100ms for rtl8211f */
    snps,reset-delays-us = <0 20000 100000>;

    assigned-clocks = <&cru SCLK_GMAC0_RX_TX>, <&cru SCLK_GMAC0>;
    assigned-clock-parents = <&cru SCLK_GMAC0_RGMII_SPEED>, <&gmac0_clkin>;
    assigned-clock-rates = <0>, <125000000>;

    pinctrl-names = "default";
     pinctrl-0 = <&gmac0_miim
                  &gmac0_tx_bus2
                  &gmac0_rx_bus2
                  &gmac0_rgmii_clk
                  &gmac0_rgmii_bus
                &gmac0_clkinout>;


    tx_delay = <0x3c>;
    rx_delay = <0x2f>;

    phy-handle = <&rgmii_phy0>;
```

```
        status = "okay";
    };

    &gmac1 {      //eth1
        phy-mode = "rgmii";
        clock_in_out = "input";
        snps,reset-gpio = <&gpio3 RK_PB0 GPIO_ACTIVE_LOW>;
        snps,reset-active-low;
        /* Reset time is 20ms, 100ms for rtl8211f */
        snps,reset-delays-us = <0 20000 100000>;

        assigned-clocks = <&cru SCLK_GMAC1_RX_TX>, <&cru SCLK_GMAC1>;
        assigned-clock-parents = <&cru SCLK_GMAC1_RGMII_SPEED>, <&gmac1_clkin>;
        assigned-clock-rates = <0>, <125000000>;

        pinctrl-names = "default";
        pinctrl-0 = <&gmac1m1_miim
                     &gmac1m1_tx_bus2
                     &gmac1m1_rx_bus2
                     &gmac1m1_rgmii_clk
                     &gmac1m1_rgmii_bus
                  &gmac1m1_clkinout>;

        tx_delay = <0x4f>;
        rx_delay = <0x26>;

        phy-handle = <&rgmii_phy1>;
        status = "okay";
    };
```

配置完毕之后，重新编译内核源码，烧写内核镜像，然后 ping 百度，发现网络可以使用。

7.2 修改网口灯状态

在经过上个小节的配置，以太网已经配置成功了，测试发现上网时，RJ45 只有一个黄灯在闪烁，但是我们需要的效果是一个 led 闪烁表示 active 状态，一个 led 常亮表示 link 状态。

修改内核源码/drivers/net/ethernet/stmicro/stmmac/stmmac_main.c 文件，蓝色代码是修改增加的内容

```
#define STMMAC_ALIGN(x)          ALIGN(ALIGN(x, SMP_CACHE_BYTES), 16)
#define TSO_MAX_BUFF_SIZE (SZ_16K - 1)
#define RTL_8211F_PHY_ID            0x001cc916
#define RTL_8211F_PHY_ID_MASK       0x001fffff
#define RTL_8211F_PAGE_SELECT       0x1f
#define RTL_8211F_LCR_ADDR          0x10
#define RTL_8211F_EEELCR_ADDR       0x11

 /* Module parameters */
 #define TX_TIMEO      5000
@@ -4266,6 +4271,40 @@ static int stmmac_hw_init(struct stmmac_priv *priv)
    return 0;
 }

static int phy_rtl8211f_led_fixup(struct phy_device *phydev)
{
        u32 val, val2;

        pr_info("terry in : %s\n", __func__);

        /*switch to page0xd04*/
        phy_write(phydev, RTL_8211F_PAGE_SELECT, 0xd04);

        /*set led1(green) Link 10/100/1000M, and set led2(yellow) Link
10/100/1000M+Active*/
        val = phy_read(phydev, RTL_8211F_LCR_ADDR);
        val |= (1<<5);
        val |= (1<<8);
        val &= (~(1<<9));
        val |= (1<<10);
        val |= (1<<11);
        phy_write(phydev, RTL_8211F_LCR_ADDR, val);

        /*set led1(green) EEE LED function disabled so it can keep on when linked*/
        val2 = phy_read(phydev, RTL_8211F_EEELCR_ADDR);
        val2 &= (~(1<<2));
        phy_write(phydev, RTL_8211F_EEELCR_ADDR, val2);

        /*switch back to page0*/
        phy_write(phydev,RTL_8211F_PAGE_SELECT, 0xa42);
```

```
                return 0;
        }

         /**
          * stmmac_dvr_probe
          * @device: device pointer
        @@ -4458,7 +4497,11 @@ int stmmac_dvr_probe(struct device *device,
                netdev_warn(priv->dev, "%s: failed debugFS registration\n",
                            __func__);
         #endif

                /* register the PHY board fixup */
            ret = phy_register_fixup_for_uid(RTL_8211F_PHY_ID, RTL_8211F_PHY_ID_MASK,
phy_rtl8211f_led_fixup);
            if (ret) {
                            dev_warn(priv->device, "Cannot register PHY board fixup, terry in :%s.\n",
__func__);
            }
            return ret;

         error_netdev_register:
```

第 8 章 Camera 使用

8.1 MIPI_CSI 用法

iTOP-RK3568 开发板平台仅有一个标准物理 mipi csi2 dphy，可以工作在俩个模式：full mode 和 split mode，拆分为 csi2_dphy0/csi2_dphy1/csi2_dphy2 三个逻辑 dphy

- Full Mode：仅使用 csi2_dphy0, csi2_dphy0 与 csi2_dphy1/csi2_dphy2 互斥，不可同时使用。data lane 最大 4 lanes。最大速率 2.5Gbps/lane;
- Split Mode: 仅使用 csi2_dphy1 和 csi2_dphy2, 与 csi2_dphy0 互斥,不可同时使用;csi2_dphy1 和 csi2_dphy2 可同时使用;csi2_dphy1 和 csi2_dphy2 各自的 data lane 最大是 2 lanes;csi2_dphy1 对应物理 dphy 的 lane0/lane1;
  csi2_dphy2 对应物理 dphy 的 lane2/lane3;最大速率 2.5Gbps/lane

Option	Sensor Lane	Interface
Full mode	sensor1 x 4Lane	MIPI_CSI_RX_D0-3 MIPI_CSI_RX_CLK0
Split mode	Sensor1 x 2Lane + Sensor2 x 2Lane	MIPI_CSI_RX_D0-1 MIPI_CSI_RX_CLK0
		MIPI_CSI_RX_D2-3 MIPI_CSI_RX_CLK1

单目摄像头我们可以配置 full mode，如果使用双目摄像头，我们可以配置 split mode

8.2 Full mode 配置

链接关系：sensor->csi2_dphy0-isp

8.2.1 设备树配置

首先我们需要根据开发板原理图的 MIPI CSI 接口，找到摄像头是挂载在哪个 i2c 总线上，然后在对应的 i2c 节点配置摄像头节点，正确配置好摄像头模组的 i2c 设备地址，引脚等属性。如下所示，是 ov13850 摄像头

```
&i2c2 {
    status = "okay";
    pinctrl-0 = <&i2c2m1_xfer>;
```

```
    ov13850: ov13850@10 {
        status = "okay";
                compatible = "ovti,ov13850";
                reg = <0x10>;
                clocks = <&cru CLK_CIF_OUT>;
                clock-names = "xvclk";
                power-domains = <&power RK3568_PD_VI>;
                pinctrl-names = "default";
                pinctrl-0 = <&cif_clk>;
                reset-gpios = <&gpio3 RK_PD4 GPIO_ACTIVE_HIGH>;
                pwdn-gpios = <&gpio3 RK_PD5 GPIO_ACTIVE_HIGH>;
                rockchip,camera-module-index = <0>;
                rockchip,camera-module-facing = "back";
                rockchip,camera-module-name = "ZC-OV13850R2A-V1";
                rockchip,camera-module-lens-name = "Largan-50064B31";
                port {
                        ov13850_out: endpoint {
                                remote-endpoint = <&mipi_in_ucam3>;
                                data-lanes = <1 2 3 4>;
                        };
                };
    };

};
```

csi2_dphy0 与 csi2_dphy1/csi2_dphy2 互斥，不可以同时使用，还需要使能 csi2_dphy_hw 节点。

```
&csi2_dphy0 {
    status = "okay";

    ports {
        #address-cells = <1>;
```

```
#size-cells = <0>;
port@0 {
    reg = <0>;
    #address-cells = <1>;
    #size-cells = <0>;

    mipi_in_ucam0: endpoint@1 {
        reg = <1>;
        remote-endpoint = <&ucam_out0>;
        data-lanes = <1 2 3 4>;
    };
    mipi_in_ucam1: endpoint@2 {
        reg = <2>;
        remote-endpoint = <&gc8034_out>;
        data-lanes = <1 2 3 4>;
    };
    mipi_in_ucam2: endpoint@3 {
        reg = <3>;
        remote-endpoint = <&ov5695_out>;
        data-lanes = <1 2>;
    };
    mipi_in_ucam3: endpoint@4 {
            reg = <4>;
            remote-endpoint = <&ov13850_out>;
            data-lanes = <1 2 3 4>;
      };


};
port@1 {
    reg = <1>;
    #address-cells = <1>;
    #size-cells = <0>;
```

```
                csidphy_out: endpoint@0 {
                        reg = <0>;
                        remote-endpoint = <&isp0_in>;
                };
            };
        };
    };
&csi2_dphy_hw {
    status = "okay";
};
&csi2_dphy1 {
            status = "disabled";
};

&csi2_dphy2 {
            status = "disabled";
};
```

Isp 相关配置，其中 rkisp_vir0 节点的 remote-endpoint 指向 csidphy_out

```
&rkisp {
    status = "okay";
};

&rkisp_mmu {
    status = "okay";
};

&rkisp_vir0 {
    status = "okay";

    port {
        #address-cells = <1>;
        #size-cells = <0>;

        isp0_in: endpoint@0 {
```

```
                reg = <0>;
                remote-endpoint = <&csidphy_out>;
            };
        };
    };
```

8.3 Split Mode 配置

链接关系：

Sensor1->csi_dphy1->isp_vir0

Sensor2->csi_dphy2->mipi_csi2->vicap->isp_vir1

8.3.1 设备树配置

我们需要根据板子原理图的 MIPI CSI 接口找到俩个 sensor 是挂在哪个 i2c 总线上，然后在对应的 i2c 节点配置俩个 camera 节点，正确配置 camera 模组的 i2c 设备地址，引脚等属性。

8.4 软件相关目录

Linux Kernel-4.19

|-- arch/arm/boot/dts #DTS 配置文件

|-- drivers/phy/rockchip

|-- phy-rockchip-mipi-rx.c #mipi dphy 驱动

|-- phy-rockchip-csi2-dphy-common.h

|-- phy-rockchip-csi2-dphy-hw.c

|-- phy-rockchip-csi2-dphy.c

|-- drivers/media

|-- platform/rockchip/cif #RKCIF 驱动

|-- platform/rockchip/isp #RKISP 驱动

|-- dev #包含 probe、异步注册、clock、pipeline、 iommu 及 media/v4l2 framework

|-- capture #包含 mp/sp/rawwr 的配置及 vb2,帧中断处理

|-- dmarx #包含 rawrd 的配置及 vb2,帧中断处理

|-- isp_params #3A 相关参数设置

|-- isp_stats #3A 相关统计

|-- isp_mipi_luma #mipi 数据亮度统计

|-- regs #寄存器相关的读写操作

|-- rkisp #isp subdev 和 entity 注册

|-- csi #csi subdev 和 mipi 配置

|-- bridge #bridge subdev,isp 和 ispp 交互桥梁

|-- platform/rockchip/ispp #rkispp 驱动

|-- dev #包含 probe、异步注册、clock、pipeline、 iommu 及 media/v4l2 framework

|-- stream #包含 4 路 video 输出的配置及 vb2,帧中断处理

|-- rkispp #ispp subdev 和 entity 注册

|-- params #TNR/NR/SHP/FEC/ORB 参数设置

|-- stats #ORB 统计信息

|-- i2

第 9 章 PWM 使用

配置 PWM 主要有以下三大步骤：配置 PWM 内核驱动，配置 PWM 设备树节点，控制 PWM 设备。

9.1 配置设备树

DTS 配置参考文档 Documentation/devicetree/bindings/pwm/pwm.txt，设备树配置参考如下所示：

```
bl: backlight {
    pwms = <&pwm 0 25000 PWM_POLARITY_INVERTED>;
    pwm-names = "backlight";
};
```

在上述代码中，几个主要参数需要说明一下。

参数一：表示 index，一般是 0，因为 rockchip pwm 每个 chip 只有一个

参数二：表示 pwm 输出波形的时间周期，单位是 ns，上面代码中配置的是 25000 表示想要得到的 PWM 输出周期是 40KHZ。

参数三：表示极性，是可选的参数，上述配置为负极性。

9.2 配置内核驱动

PWM 驱动文件在内核源码 drivers/pwm/pwm-rockchip.c 下。

用户可以在其他的驱动文件中使用 PWM 生成 PWM 节点。具体方法如下所示：

步骤一：在要使用 PWM 控制的设备驱动文件中包含 PWM 的头文件。

```
#include <linux/pwm.h>
```

该头文件主要包含 PWM 的函数接口.

步骤二：申请 PWM

我们使用以下函数来申请 PWM

```
struct pwm_device *pwm_request(int pwm_id, const char *label);
```

例如 PWM 申请如下所示：

```
struct pwm_device * pwm0 = NULL;
pwm0 = pwm_request(1, “test-pwm”);
```

步骤三：配置 PWM

我们使用以下函数来配置 PWM

```
int pwm_config(struct pwm_device *pwm, int duty_ns, int period_ns);
```

配置 PWM 占空比，如下所示：

```
pwm_config(pwm0, 500000, 1000000);
```

步骤四：使能 PWM

我们使用以下函数来使能 PWM

```
int pwm_enable(struct pwm_device *pwm);
```

使能 PWM，如下所示：

```
pwm_enable(pwm0);
```

步骤五：使用 pwm_free 函数释放所申请的 PWM

```
void pwm_free(struct pwm_device *pwm);
```

释放 PWM，如下所示：

```
pwm_free(pwm0);
```

步骤六：使用 pwm_disable 函数禁用 PWM

```
void pwm_disable(struct pwm_device *pwm);
```

禁用 PWM，如下所示：

```
pwm_disable(pwm0);
```

9.3 PWM 调试

使用以下命令查看注册是否成功，成功则返回接口名和寄存器地址，如下图所示：

```
cat /sys/kernel/debug/pwm
```

```
root@rk3568-buildroot:/#
root@rk3568-buildroot:/# cat /sys/kernel/debug/pwm
platform/fe6e0020.pwm, 1 PWM device
 pwm-0   ((null)              ): period: 0 ns duty: 0 ns polarity: normal

platform/fe6e0010.pwm, 1 PWM device
 pwm-0   (backlight           ): requested enabled period: 25000 ns duty: 19607 ns polarity: normal

platform/fe6e0000.pwm, 1 PWM device
 pwm-0   ((null)              ): period: 0 ns duty: 0 ns polarity: normal
root@rk3568-buildroot:/#
root@rk3568-buildroot:/#
```

9.4 PWM 使用

PWM 提供了用户层的接口，在 /sys/class/pwm/ 节点下面，PWM 驱动加载成功后，会在 /sys/class/pwm/ 目录下产生 pwmchipX 目录；如下图所示：

```
root@rk3568-buildroot:/# cd /sys/class/pwm/
root@rk3568-buildroot:/sys/class/pwm# ls
pwmchip0  pwmchip1  pwmchip2
root@rk3568-buildroot:/sys/class/pwm#
root@rk3568-buildroot:/sys/class/pwm#
root@rk3568-buildroot:/sys/class/pwm#
```

可以看到多个 pwmchipx，x 为编号数字，我的开发板上有 3 个 pwmchipx，说明设备树中打开了 3 个 PWM，系统会将已经打开的 PWM 按照 PWM 编号顺序放置在/sys/class/pwm 目录，此处 pwmchip0 对应的是 PWM3，pwmchip1 对应 PWM4，pwmchip2 对应 PWM5，其中 PWM5 为 mipi 屏的背光，我们进入到 pwmchip2 目录，如下图所示：

```
root@rk3568-buildroot:/sys/class/pwm# cd pwmchip2/
root@rk3568-buildroot:/sys/class/pwm/pwmchip2#
root@rk3568-buildroot:/sys/class/pwm/pwmchip2# ls
device  export  npwm  power  subsystem  uevent  unexport
root@rk3568-buildroot:/sys/class/pwm/pwmchip2#
```

该目录下有七个文件，比较重要的是 export，npwm，和 unexport 三个文件

npwm 是一个只读文件，可以通过该文件读到 PWM 控制器下有多少路 PWM 输出，当前我的系统读出来的值是 1，如下图所示：

```
root@rk3568-buildroot:/sys/class/pwm/pwmchip2#
root@rk3568-buildroot:/sys/class/pwm/pwmchip2# cat npwm
1
root@rk3568-buildroot:/sys/class/pwm/pwmchip2#
```

export 用于导出一个新的可操作文件，比如 echo 0 > export 会在 pwmchip0 目录下新建一个 pwm0 文件夹（由于上面已经知晓 npwm 为 1，所以只能导出 pwm0，不存在 pwm1…）

```
root@rk3568-buildroot:/sys/class/pwm/pwmchip2#
root@rk3568-buildroot:/sys/class/pwm/pwmchip2# echo 0 > export
root@rk3568-buildroot:/sys/class/pwm/pwmchip2# ls
device  export  npwm  power  pwm0  subsystem  uevent  unexport
root@rk3568-buildroot:/sys/class/pwm/pwmchip2#
```

unexport 是 export 的反向操作，用于删除已经导出的 pwm 设备，它们都是只写文件，

进入 pwm0 目录，又能看到八个文件，比较重要的是 polarity，period，duty_cycle 和 enable，如下图所示：

```
root@rk3568-buildroot:/sys/class/pwm/pwmchip2#
root@rk3568-buildroot:/sys/class/pwm/pwmchip2# cd pwm0/
root@rk3568-buildroot:/sys/class/pwm/pwmchip2/pwm0# ls
capture  duty_cycle  enable  output_type  period  polarity  power  uevent
root@rk3568-buildroot:/sys/class/pwm/pwmchip2/pwm0#
root@rk3568-buildroot:/sys/class/pwm/pwmchip2/pwm0#
root@rk3568-buildroot:/sys/class/pwm/pwmchip2/pwm0#
```

- duty_cycle 用于配置 PWM 占空比，单位为 ns。
- enable 为 1 时使能 PWM 输出，为 0 时关闭 PWM 输出。
- period 用于配置 PWM 周期，单位为 ns。
- polarity 用于配置 PWM 极性，“normal”（普通输出），“inversed”（反转输出）。

注：必须先设置 period 周期后才可以设置其他属性。

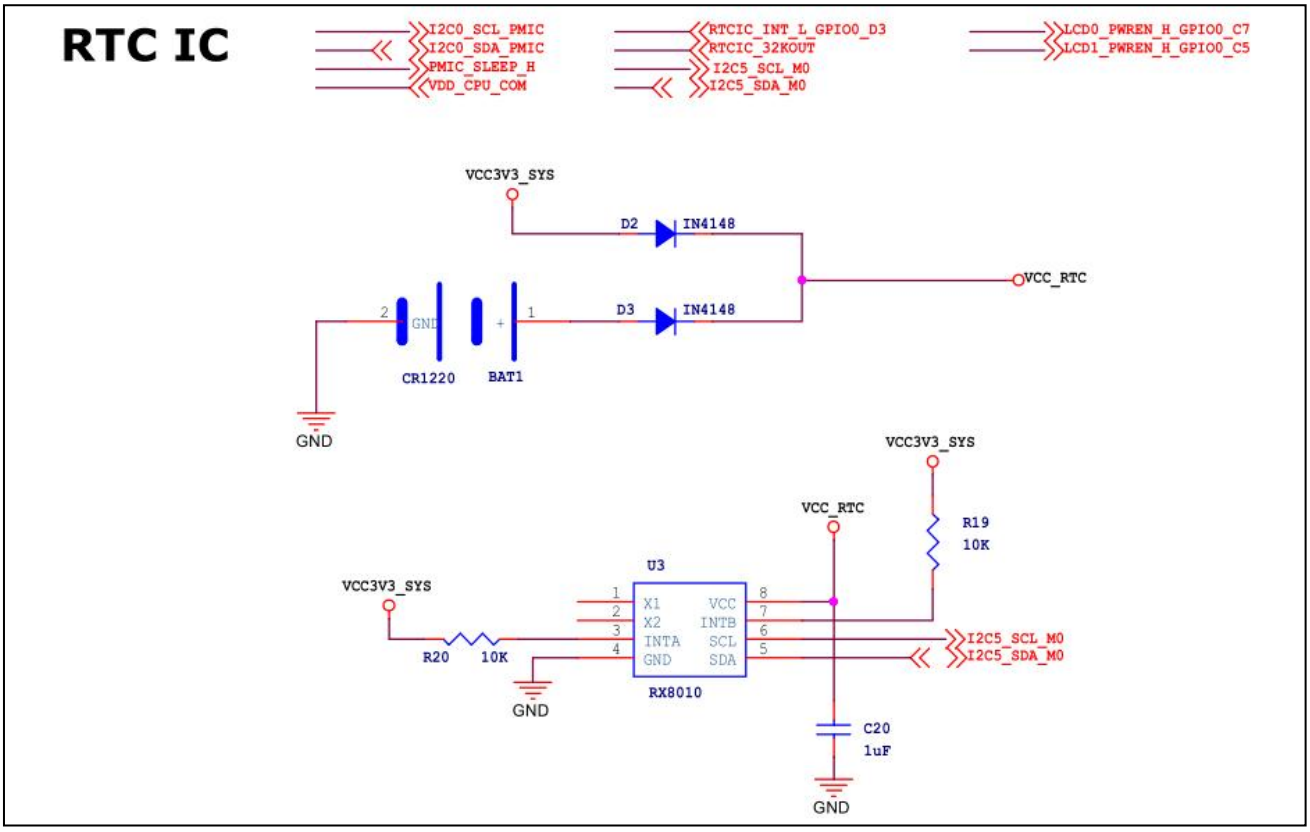
9.5 常见问题

PWM 无法注册成功：

- dts 配置文件是否打开对应的 PWM。
- PWM 所在的 IO 口是否被其他资源占用，可以根据报错的返回值去查看原因。

第 10 章 RTC 使用

RTC 也就是实时时钟，RTC 为操作系统提供了一个可靠的时间，并且在断电的情况下，RTC 实时时钟也可以通过电池供电，一直运行下去。 iTOP-RK3568 开发板采用 RX8010 作为 RTC（real time clock），RTC 原理图如下图所示：



10.1 设备树驱动配置

设备树 kernel/arch/arm64/boot/dts/rockchip/rk3568-evb.dtsi 配置如下所示：

```
&i2c5 {
    status = "okay";
    rx8010: rx8010@32 {
            compatible = "epson,rx8010";
            reg = <0x32>;
            status = "okay";
        #clock-cells = <0>;
     };
```

```
};
```

驱动参考 kernel/drivers/rtc/rtc-rx8010.c

10.2 接口使用

Linux 提供了三种用户接口空间调用接口。在 iTOP-RK3568 开发板中对应的路径是

- SYSFS 接口：/sys/class/rtc/rtc0/
- PROCFS 接口：/proc/driver/rtc
- IOCTL 接口：/dev/rtc0

我们可以使用 cat 和 echo 操作/sys/class/rtc/rtc0/ 下面的接口。

比如我们查看当前 RTC 的日期和时间，输入以下命令：

```
cat /sys/class/rtc/rtc0/date
cat /sys/class/rtc/rtc0/time
```

```
root@rk3568-buildroot:/# cat /sys/class/rtc/rtc0/date
2023-07-20
root@rk3568-buildroot:/# cat /sys/class/rtc/rtc0/time
02:28:49
root@rk3568-buildroot:/#
```

设置开机时间，如设置 120 秒后开机，输入以下命令

```
echo 120 > /sys/class/rtc/rtc0/wakealarm
```

查看开机时间，输入以下命令

```
cat /sys/class/rtc/rtc0/wakealarm
```

关机输入以下命令

```
reboot -p
```

使用 PROCFS 接口打印 RTC 相关的信息，输入以下命令

```
cat /proc/driver/rtc
```

```
root@rk3568-buildroot:/# cat /proc/driver/rtc
rtc_time        : 02:41:18
rtc_date        : 2023-07-20
24hr            : yes
root@rk3568-buildroot:/#
```

注：如果开发板上电后，时间不同步，首先要检查下 RTC 电池有没有正确接入。

第 11 章 CAN 使用

Controller Area Network（CAN）是一种用于在不同的电子控制单元（ECU）之间传输数据的串行通信协议。CAN 总线具有以下特点：

- 高速通信：CAN 总线的通信速率可达 1 Mbps，适用于实时控制应用。
- 高容错性：内建强大的检错和处理机制，能够在嘈杂的电磁环境中可靠工作。
- 广播特性：所有节点可以侦听所有传输的报文，但可以通过本地过滤功能实现报文选择性捕获。
- 帧结构：每个 CAN 报文包含标识符、控制位、数据字段等组成部分。
- ISO 标准：有多个 ISO 标准，如 ISO11898 和 ISO11519，定义了不同速率和特性的 CAN 通信标准。

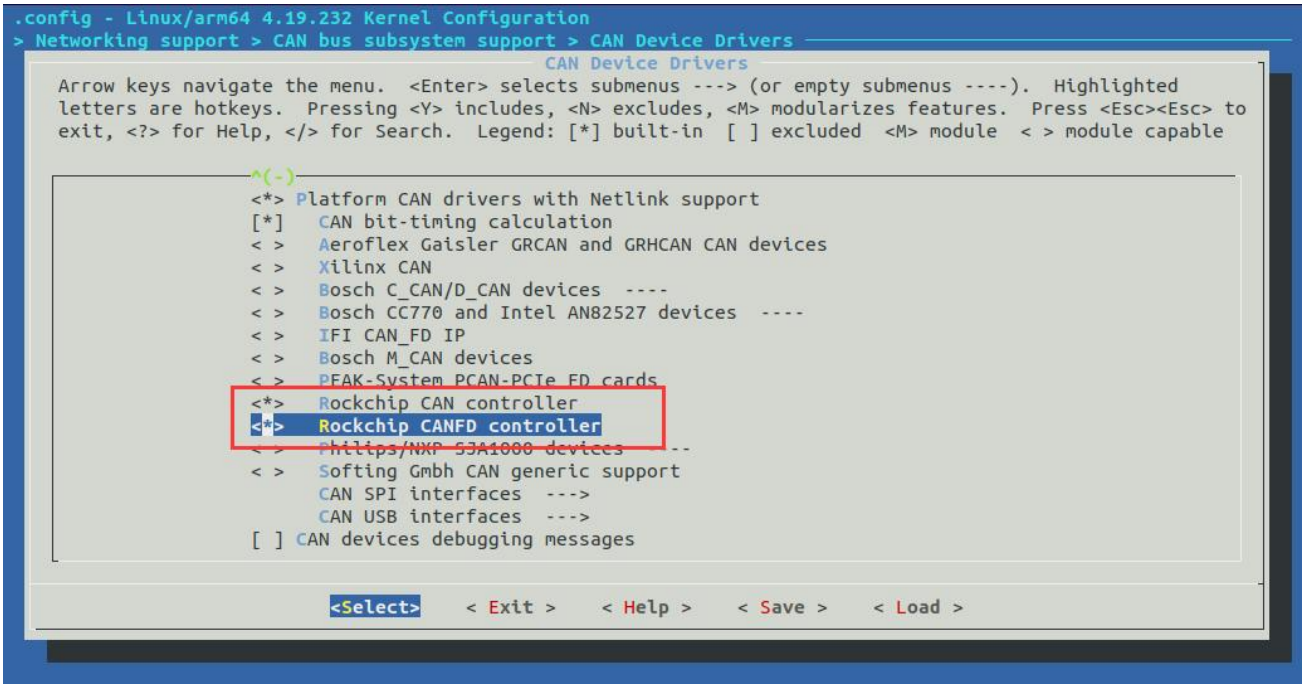
与 I2C,SPI 等同步通讯方式不同，CAN 通讯是异步通讯，也就是没有时钟信号线来保持信号接收同步，也就是所说的半双工，无法同时发送与接收，在同一时刻，只能有一个节点发送数据，其余节点都只能接收数据。但是在 CAN 协议中，我们可以挂载多个 节点 ，通过总线来实现节点通讯，与其他协议不同的是，不对节点的地址进行编码，而是对节点的数据内容进行编码。理论上节点个数不受限制。

11.1 CAN 驱动配置

can 控制器驱动程序在 /kernel/drivers/net/can/rockchip/rockchip_can.c 和 drivers/net/can/rockchip/rockchip_canfd.c

```
topeet@ubuntu:~/Linux/rk356x_linux/kernel$ cd drivers/net/can/rockchip/
topeet@ubuntu:~/Linux/rk356x_linux/kernel/drivers/net/can/rockchip$ ls
built-in.a  Kconfig  Makefile  modules.builtin  modules.order  rockchip_can.c  rockchip_canfd.c
topeet@ubuntu:~/Linux/rk356x_linux/kernel/drivers/net/can/rockchip$
topeet@ubuntu:~/Linux/rk356x_linux/kernel/drivers/net/can/rockchip$
topeet@ubuntu:~/Linux/rk356x_linux/kernel/drivers/net/can/rockchip$
```

linux 内核配置项点，在 Networking support > CAN bus subsystem support > CAN Device Drivers 下打开 can drivers 配置项点，默认迅为提供的源码 CAN 和 CANFD 都是支持的，如下图所示：



11.2 设备树节点配置

设备树 kernel/arch/arm64/boot/dts/rockchip/rk3568-evb1-ddr4-v10-linux.dts 配置如下所示：

```
&can1 {
        status = "okay";
        compatible = "rockchip,canfd-1.0";
        assigned-clocks = <&cru CLK_CAN1>;
        assigned-clock-rates = <150000000>;
        pinctrl-names = "default";
        pinctrl-0 = <&can1m1_pins>;
};
```

compatible = "rockchip,canfd-1.0" ，rockchip,canfd-1.0 用来匹配 can 控制器驱动,如果只是简单的 can 控制器收发的话采用 drivers/net/can/rockchip/rockchip_can.c（将 compatible 改为“rockchip,can-1.0”即可）

assigned-clock-rates 用来配置 can 的始终频率，如果 CAN 的波特率为 1M，建议修改 CAN 时钟为 300M，低于 1M 的波特率时钟设置为 200M 更稳定。

pinctrl 配置：根据实际板卡连接情况配置 can_h 和 can_l 的 iomux 作为 can 功能使用。

11.3 canutils 工具配置使用

canutils 是常用的 CAN 通信测试工具包，内含 5 个独立的程序：canconfig、candump、canecho、cansend、cansequence。这几个程序的功能简述如下：

- canconfig ：用于配置 CAN 总线接口的参数，主要是波特率和模式。
- candump ：从 CAN 总线接口接收数据并以十六进制形式打印到标准输出，也可以输出到指定文件。
- canecho ：把从 CAN 总线接口接收到的所有数据重新发送到 CAN 总线接口。
- cansend ：往指定的 CAN 总线接口发送指定的数据。
- cansequence ：往指定的 CAN 总线接口自动重复递增数字，也可以指定接收模式并校验检查接收的递增数字。
- ip ：CAN 波特率、功能等配置。

测试指令：

1、can 启动：ip link set can0 up

2、can 关闭：ip link set can0 down

3、设置波特率：ip link set can0 type can bitrate 1000000 dbitrate 3000000 fd on

4、显示 can0 详细信息：ip -details link show can0

5、can 数据发送格式 cansend <device> <can_frame>：

```
<can_frame>:
<can_id>#{data}              for 'classic' CAN 2.0 data frames
<can_id>#R{len}              for 'classic' CAN 2.0 data frames
<can_id>##<flags>{data}    for CAN FD frames

<can_id>:
  3 (SFF) or 8 (EFF) hex chars
{data}:
  0..8 (0..64 CAN FD) ASCII hex-values (optionally separated by '.')
{len}:
  an optional 0..8 value as RTR frames can contain a valid dlc field
<flags>:
  a single ASCII Hex value (0 .. F) which defines canfd_frame.flags

Examples:
   5A1#11.2233.44556677.88 / 123#DEADBEEF / 5AA# / 123##1 / 213##311223344 /
   1F334455#1122334455667788 / 123#R / 00000123#R3
```

6、can 接收：candump can0。

11.4 CAN 应用测试程序

配套资料在“iTOP-3568 开发板\02_【iTOP-RK3568 开发板】开发资料\09_系统外设功能测试\07_外设接口配置配套资料\03_CAN 应用测试配套资料”目录下。

can 应用测试程序如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <net/if.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <sys/ioctl.h>
#include <linux/can.h>
#include <linux/can/raw.h>

void print_can_frame(struct can_frame *frame) {
    printf("CAN ID: 0x%X\n", frame->can_id);
    printf("Data Length: %d\n", frame->can_dlc);
    printf("Data: ");
    for (int i = 0; i < frame->can_dlc; ++i) {
        printf("%02X ", frame->data[i]);
    }
    printf("\n");
}

int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Usage: %s <CAN interface> <operation: recv or send>\n", argv[0]);
        return 1;
    }

    const char *can_interface = argv[1];
    const char *operation = argv[2];
```

```
// Create a socket
int s = socket(PF_CAN, SOCK_RAW, CAN_RAW);
if (s == -1) {
    perror("socket");
    return 1;
}

// Get the interface index
struct ifreq ifr;
strcpy(ifr.ifr_name, can_interface);
ioctl(s, SIOCGIFINDEX, &ifr);

// Bind the socket to the CAN interface
struct sockaddr_can addr;
addr.can_family = AF_CAN;
addr.can_ifindex = ifr.ifr_ifindex;
if (bind(s, (struct sockaddr *)&addr, sizeof(addr)) == -1) {
    perror("bind");
    close(s);
    return 1;
}

if (strcmp(operation, "recv") == 0) {
    // Receive mode
    struct can_frame frame;
    while (1) {
        // Receive a CAN frame
        ssize_t nbytes = read(s, &frame, sizeof(struct can_frame));
        if (nbytes < 0) {
            perror("read");
            close(s);
            return 1;
        }
```

```
            // Print the received CAN frame
            printf("Received CAN Frame:\n");
            print_can_frame(&frame);
        }
    } else if (strcmp(operation, "send") == 0) {
        // Send mode
        struct can_frame frame;
        frame.can_id = 0x123;    // Example CAN ID
        frame.can_dlc = 8;       // Example data length
        memset(frame.data, 0xAA, 8);    // Example data

        // Send the CAN frame
        ssize_t nbytes = write(s, &frame, sizeof(struct can_frame));
        if (nbytes < 0) {
            perror("write");
            close(s);
            return 1;
        }
        printf("Sent CAN Frame:\n");
        print_can_frame(&frame);
    } else {
        printf("Invalid operation. Use 'recv' or 'send'.\n");
    }

    close(s);
    return 0;
}
```

代码详解如下：

- print_can_frame 函数： 这个函数用于打印 CAN 帧的信息，包括 CAN ID、数据长度和数据内容。
- main 函数： 程序的主要入口。
- 命令行参数检查： 程序首先检查命令行参数的数量。需要两个参数：CAN 接口的名称（例如：can0）和操作（recv 或 send）。

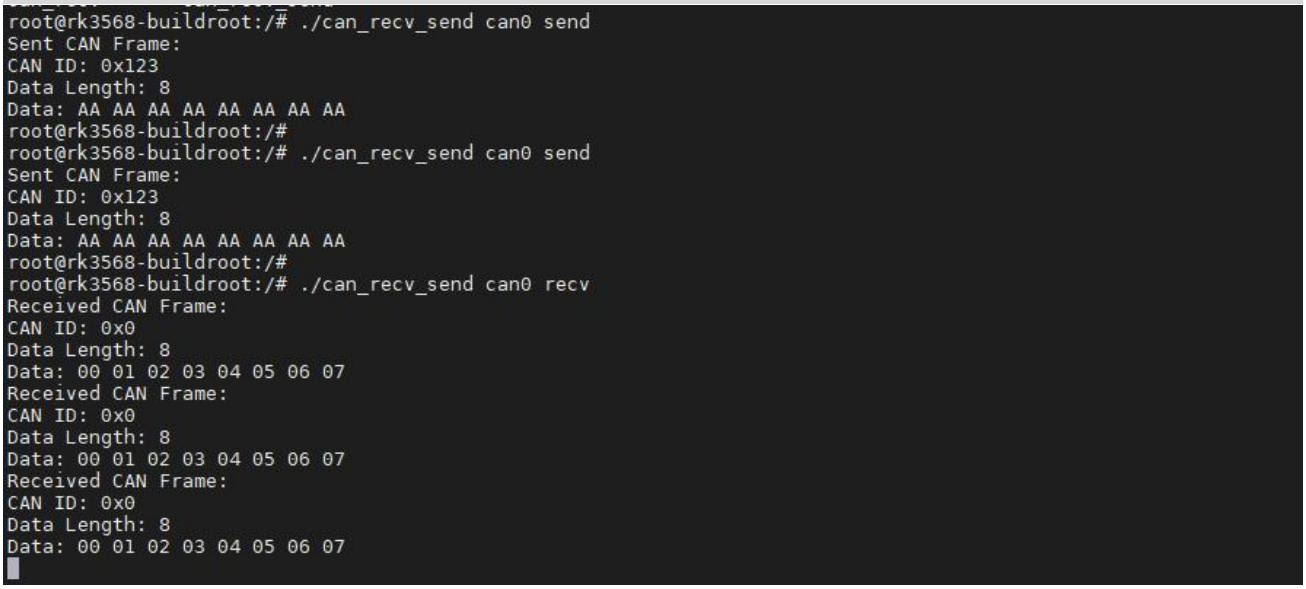
- 创建 Socket： 通过 socket 函数创建一个 CAN socket。PF_CAN 表示使用 CAN 协议族，SOCK_RAW 表示原始套接字，CAN_RAW 表示使用 CAN 原始协议。
- 获取接口索引： 使用 ioctl 函数和 SIOCGIFINDEX 参数来获取指定 CAN 接口的索引。
- 绑定 Socket： 使用 bind 函数将 Socket 绑定到指定的 CAN 接口。
- 接收模式： 如果操作是 "recv"，则进入接收模式。在一个无限循环中，使用 read 函数从 Socket 中读取 CAN 帧，并通过调用 print_can_frame 函数打印接收到的 CAN 帧的信息。
- 发送模式： 如果操作是 "send"，则进入发送模式。首先创建一个 CAN 帧，设置 CAN ID、数据长度和数据内容，然后使用 write 函数将 CAN 帧写入 Socket 中。
- 关闭 Socket： 最后，通过 close 函数关闭 Socket。

使用以下命令设置 can0 仲裁段 1M 波特率，数据段 3M 波特率如下图所示

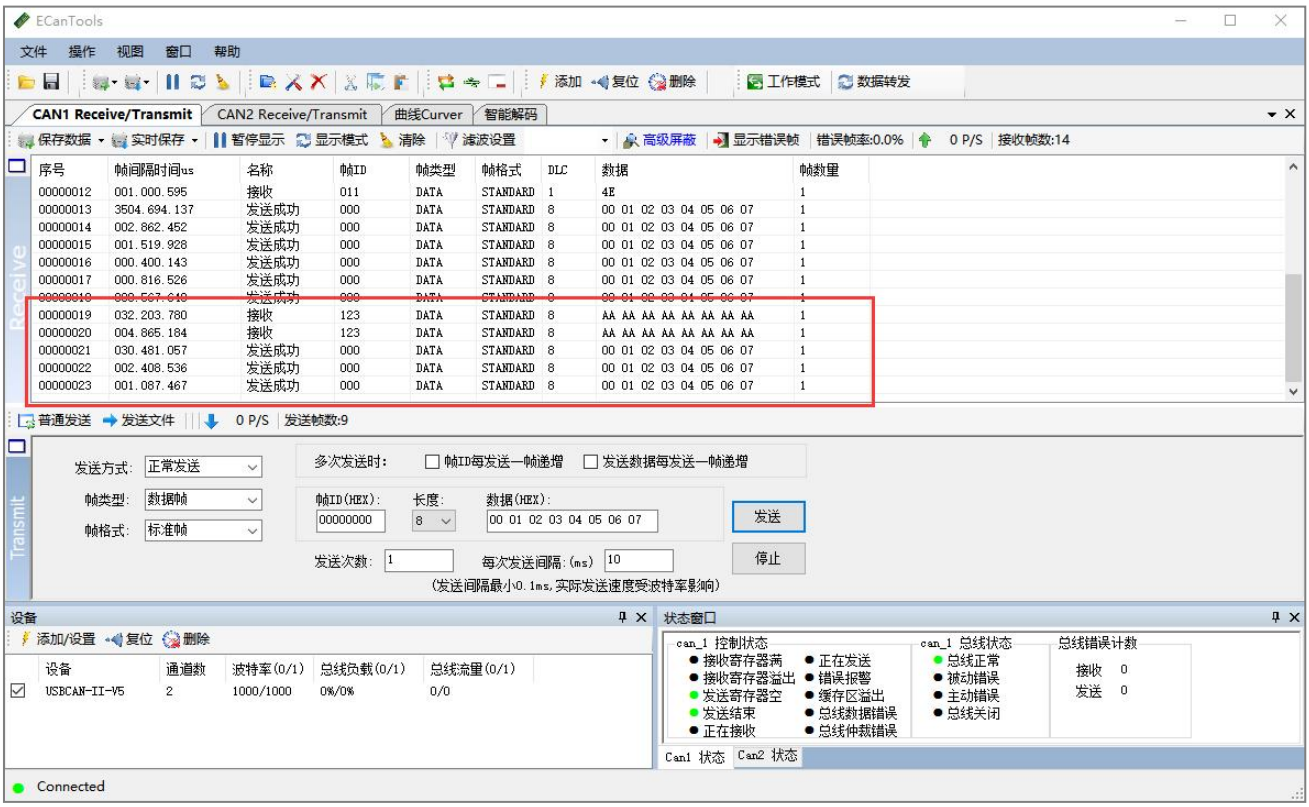
```
ip link set can0 down
ip link set can0 type can bitrate 1000000 dbitrate 3000000 fd on
ip link set can0 up
```

使用以下命令运行程序测试，如下图所示：

```
./can_recv_send can0 send
./can_recv_send can0 recv
```



```
root@rk3568-buildroot:/# ./can_recv_send can0 send
Sent CAN Frame:
CAN ID: 0x123
Data Length: 8
Data: AA AA AA AA AA AA AA AA
root@rk3568-buildroot:/#
root@rk3568-buildroot:/# ./can_recv_send can0 send
Sent CAN Frame:
CAN ID: 0x123
Data Length: 8
Data: AA AA AA AA AA AA AA AA
root@rk3568-buildroot:/#
root@rk3568-buildroot:/# ./can_recv_send can0 recv
Received CAN Frame:
CAN ID: 0x0
Data Length: 8
Data: 00 01 02 03 04 05 06 07
Received CAN Frame:
CAN ID: 0x0
Data Length: 8
Data: 00 01 02 03 04 05 06 07
Received CAN Frame:
CAN ID: 0x0
Data Length: 8
Data: 00 01 02 03 04 05 06 07
```


ECanTools
文件 操作 视图 窗口 帮助
添加 复位 删除 工作模式 数据转发
CAN1 Receive/Transmit
CAN2 Receive/Transmit
曲线Curver
智能解码
保存数据 实时保存 暂停显示 显示模式 清除 滤波设置 高级屏蔽 显示错误帧 错误帧率:0.0% 0 P/S 接收帧数:14
序号 帧间隔时间us 名称 帧ID 帧类型 帧格式 DLC 数据 帧数量
00000012 001.000.595 接收 011 DATA STANDARD 1 4E 1
00000013 3504.694.137 发送成功 000 DATA STANDARD 8 00 01 02 03 04 05 06 07 1
00000014 002.862.452 发送成功 000 DATA STANDARD 8 00 01 02 03 04 05 06 07 1
00000015 001.519.928 发送成功 000 DATA STANDARD 8 00 01 02 03 04 05 06 07 1
00000016 000.400.143 发送成功 000 DATA STANDARD 8 00 01 02 03 04 05 06 07 1
00000017 000.816.526 发送成功 000 DATA STANDARD 8 00 01 02 03 04 05 06 07 1
00000019 032.203.780 接收 123 DATA STANDARD 8 AA AA AA AA AA AA AA AA 1
00000020 004.865.184 接收 123 DATA STANDARD 8 AA AA AA AA AA AA AA AA 1
00000021 030.481.057 发送成功 000 DATA STANDARD 8 00 01 02 03 04 05 06 07 1
00000022 002.408.536 发送成功 000 DATA STANDARD 8 00 01 02 03 04 05 06 07 1
00000023 001.087.467 发送成功 000 DATA STANDARD 8 00 01 02 03 04 05 06 07 1
普通发送 发送文件 0 P/S 发送帧数:9
Transmit
发送方式: 正常发送
帧类型: 数据帧
帧格式: 标准帧
多次发送时: 帧ID每发送一帧递增 发送数据每发送一帧递增
帧ID(HEX): 00000000
长度: 8
数据(HEX): 00 01 02 03 04 05 06 07
发送
发送次数: 1
每次发送间隔:(ms) 10
停止
(发送间隔最小0.1ms,实际发送速度受波特率影响)
设备
添加/设置 复位 删除
设备 通道数 波特率(0/1) 总线负载(0/1) 总线流量(0/1)
USBCAN-II-V5 2 1000/1000 0%/0% 0/0
状态窗口
can_1 控制状态
接收寄存器满 正在发送
接收寄存器溢出 错误报警
发送寄存器空 缓存区溢出
发送结束 总线数据错误
正在接收 总线仲裁错误
can_1 总线状态
总线正常
被动错误
主动错误
总线关闭
总线错误计数
接收 0
发送 0
Can1 状态 Can2 状态
Connected